

SOC612_L3

Raphi Duerr

2026-09-22

SOC612: Data Analytics for the Social Sciences

Date: September 22, 2026

Author: R. Duerr

Lecture 3: Descriptive and exploratory data analysis

To apply some techniques for data descriptions and explorations, we use the National Health Interview Survey (2020). It contains many health-, medical-, and lifestyle-related variables. We explore the subjective health status of the sample participants, as well as some of their demographics, and lifestyle-related choices.

In a first step, we tidy the dataset, and subset our variables of interest to prepare it for our further analytic purposes.

```
library(readr)
```

```
## Warning: package 'readr' was built under R version 4.2.3
```

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.2.3
```

```
## Warning: package 'ggplot2' was built under R version 4.2.3
```

```
## Warning: package 'tibble' was built under R version 4.2.3
```

```
## Warning: package 'tidyr' was built under R version 4.2.3
```

```
## Warning: package 'purrr' was built under R version 4.2.3
```

```
## Warning: package 'dplyr' was built under R version 4.2.3
```

```
## Warning: package 'forcats' was built under R version 4.2.3
```

```
## Warning: package 'lubridate' was built under R version 4.2.3
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.3      ✓ purrr      1.0.2
## ✓ forcats    1.0.0      ✓ stringr    1.5.0
## ✓ ggplot2     3.5.1      ✓ tibble     3.2.1
## ✓ lubridate  1.9.3      ✓ tidyr      1.3.0
## — Conflicts — tidyverse_conflicts() —
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to
become errors
```

```
setwd(getwd())
data <- read_csv("nhis-20-a.csv")
```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)
```

```
## Rows: 31568 Columns: 617
## — Column specification —
## Delimiter: ","
## chr   (1): HHX
## dbl (605): URBRL, RATCAT_A, INCGRP_A, INCTCFLG_A, FAMINCTC_A, IMPINCFLG_A, ...
## lgl  (11): OGFLG_A, OPFLG_A, CHFLG_A, PRPLCOV2_C_A, STOMAAGETC_A, RECTUAGETC...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```

#subsetting
data <- select(data, health_status=PHSTAT_A,
  #lifestyle
  lfstl_vap=ECIGEV_A,lfstl_smo=SMKEV_A,lfstl_sle=SLPHOURS_A,lfstl_alc=DRKLIFE_A,
  #sociodemographics
  age=AGEP_A,gender=SEX_A,family_income=FAMINCTC_A,education=EDUC_A)

#recoding
data <- data %>%
  mutate(health_status = recode(health_status,
    `1` = "excellent",
    `2` = "very_good",
    `3` = "good",
    `4` = "fair",
    `5` = "poor",
    .default = NA_character_)) %>%
  mutate(health_status = factor(health_status,
    levels = c("poor","fair","good",
      "very_good","excellent")))

data <- data %>%
  mutate(gender = recode(gender,
    `1` = "male",
    `2` = "female",
    .default = NA_character_)) %>%
  mutate(education = case_when(
    education %in% c(0,1) ~ "primary_or_below",
    education %in% c(2,3,4,5) ~ "secondary",
    education %in% c(6,7) ~ "post_secondary",
    education %in% c(8,9,10,11) ~ "tertiary",
    TRUE ~ NA_character_)) %>%
  mutate(gender = factor(gender,
    levels = c("male","female"))) %>%
  mutate(education = factor(education,
    levels = c("primary_or_below","secondary",
      "post_secondary","tertiary")))

data <- data %>%
  mutate(lfstl_vap = recode(lfstl_vap,
    `1` = "yes",
    `2` = "no",
    .default = NA_character_)) %>%
  mutate(lfstl_smo = recode(lfstl_smo,
    `1` = "yes",
    `2` = "no",
    .default = NA_character_)) %>%
  mutate(lfstl_sle = ifelse(lfstl_sle > 24 | lfstl_sle < 1, NA, lfstl_sle)) %>%
  mutate(lfstl_alc = recode(lfstl_alc,
    `1` = "yes",
    `2` = "no",
    .default = NA_character_)) %>%
  mutate(lfstl_vap = factor(lfstl_vap, levels = c("yes", "no"))) %>%
  mutate(lfstl_smo = factor(lfstl_smo, levels = c("yes", "no"))) %>%
  mutate(lfstl_sle = as.numeric(lfstl_sle)) %>%

```

```
mutate(lfstl_alc = factor(lfstl_alc, levels = c("yes", "no")))

data <- na.omit(data)
```

Our dataset "data" contains 30,328 complete observations of 9 variables. We can take a first look at it by using `glimpse()` or `head()`.

In a further step, we can look at the aggregate measures of the dataset, using, for example, `summary()` or for more details, `describe()`

```
#glimpse, head, skim
glimpse(data)
```

```
## Rows: 30,328
## Columns: 9
## $ health_status <fct> very_good, very_good, good, excellent, good, fair, very_...
## $ lfstl_vap      <fct> no, no, yes, no, no, no, no, no, no, no, no, no, no, no, no,...
## $ lfstl_smo      <fct> no, no, yes, no, no, no, no, no, no, no, no, no, no, no, no,...
## $ lfstl_sle      <dbl> 9, 6, 9, 8, 8, 7, 8, 8, 8, 8, 7, 7, 6, 6, 6, 6, 4, 9, 4,...
## $ lfstl_alc      <fct> yes, yes, yes, yes, no, yes, yes, yes, yes, no, yes, no, yes,...
## $ age            <dbl> 85, 39, 42, 32, 85, 70, 32, 77, 71, 69, 56, 60, 62, 58, ...
## $ gender         <fct> male, male, male, male, female, female, female, female, ...
## $ family_income  <dbl> 100000, 75000, 90000, 65000, 25762, 36000, 30105, 9000, ...
## $ education      <fct> secondary, secondary, secondary, tertiary, secondary, te...
```

```
head(data, 10) #we select the first 10 observations
```

```
## # A tibble: 10 × 9
##   health_status lfstl_vap lfstl_smo lfstl_sle lfstl_alc   age gender
##   <fct>         <fct>      <fct>      <dbl> <fct>     <dbl> <fct>
## 1 very_good    no        no          9 yes      85 male
## 2 very_good    no        no          6 yes      39 male
## 3 good         yes       yes         9 yes      42 male
## 4 excellent    no        no          8 yes      32 male
## 5 good         no        no          8 no       85 female
## 6 fair         no        no          7 yes      70 female
## 7 very_good    no        no          8 yes      32 female
## 8 fair         no        no          8 yes      77 female
## 9 very_good    no        no          8 no       71 male
## 10 good        no        no          8 yes      69 female
## # i 2 more variables: family_income <dbl>, education <fct>
```

```
#summary
summary(data)
```

```
##   health_status   lfstl_vap   lfstl_smo   lfstl_sle   lfstl_alc
## poor      : 980   yes: 4393   yes:11485   Min.    : 1.000   yes:27084
## fair      : 3275   no :25935   no :18843   1st Qu.: 6.000   no : 3244
## good      : 8467
## very_good:10627
## excellent: 6979
##                                     Median : 7.000
##                                     Mean    : 7.188
##                                     3rd Qu.: 8.000
##                                     Max.    :24.000
##   age            gender   family_income   education
## Min.    :18.00   male :13954   Min.    :    0   primary_or_below: 1925
## 1st Qu.:38.00   female:16374   1st Qu.: 33866   secondary        :12387
## Median :55.00
## Mean    :53.55
## 3rd Qu.:68.00
## Max.    :99.00
##                                     Median : 64000   post_secondary   : 3984
##                                     Mean    : 80565   tertiary         :12032
##                                     3rd Qu.:107000
##                                     Max.    :250000
```

```
#describe
library(Hmisc)
```

```
## Warning: package 'Hmisc' was built under R version 4.2.3
```

```
##
## Attaching package: 'Hmisc'
```

```
## The following objects are masked from 'package:dplyr':
##
##   src, summarize
```

```
## The following objects are masked from 'package:base':
##
##   format.pval, units
```

```
describe(data)
```

```

## data
##
## 9 Variables      30328 Observations
## -----
## health_status
##      n missing distinct
##    30328      0      5
##
## Value      poor      fair      good very_good excellent
## Frequency      980      3275      8467      10627      6979
## Proportion    0.032    0.108    0.279    0.350    0.230
## -----
## lfstl_vap
##      n missing distinct
##    30328      0      2
##
## Value      yes      no
## Frequency  4393 25935
## Proportion 0.145 0.855
## -----
## lfstl_smo
##      n missing distinct
##    30328      0      2
##
## Value      yes      no
## Frequency  11485 18843
## Proportion 0.379 0.621
## -----
## lfstl_sle
##      n missing distinct      Info      Mean      Gmd      .05      .10
##    30328      0      22    0.929    7.188    1.385      5      6
##      .25      .50      .75      .90      .95
##      6      7      8      8      9
##
## lowest : 1 2 3 4 5, highest: 18 19 20 22 24
## -----
## lfstl_alc
##      n missing distinct
##    30328      0      2
##
## Value      yes      no
## Frequency  27084  3244
## Proportion 0.893 0.107
## -----
## age
##      n missing distinct      Info      Mean      Gmd      .05      .10
##    30328      0      70      1    53.55    20.89     24     28
##      .25      .50      .75      .90      .95
##      38      55      68      77      83
##
## lowest : 18 19 20 21 22, highest: 83 84 85 97 99
## -----
## gender
##      n missing distinct

```

```
##      30328      0      2
##
## Value      male female
## Frequency  13954 16374
## Proportion 0.46  0.54
## -----
## family_income
##      n missing distinct      Info      Mean      Gmd      .05      .10
##   30328      0      3504      1    80565    66582    12000    18000
##      .25      .50      .75      .90      .95
##   33866    64000    107000    175000    225000
##
## lowest :      0      1      2      5      10, highest: 245000 245499 247000 248000 25000
## -----
## education
##      n missing distinct
##   30328      0      4
##
## Value      primary_or_below      secondary      post_secondary      tertiary
## Frequency      1925      12387      3984      12032
## Proportion      0.063      0.408      0.131      0.397
## -----
```

The amount of information we get from categorical and numerical variables is different. We can also compute these measures individually, if we want to extract a specific value, or change an existing one. Functions for common point estimates like mean, median, or sd, among others, are built-in functions, while for others like the IQR-function, a package is sometimes necessary.

For the proportions of the factors, table1() (from table1 package) offers a good function.

```
#mean, sd, IQR, skewness (for numeric variables)
library(e1071)
```

```
## Warning: package 'e1071' was built under R version 4.2.3
```

```
##
## Attaching package: 'e1071'
```

```
## The following object is masked from 'package:Hmisc':
##
##      impute
```

```

age.mean <- round(mean(data$age),3)
age.sd <- round(sd(data$age),3)
age.iqr <- round(IQR(data$age),3)
age.skewness <- round(skewness(data$age),3)
age.sum <- round(sum(data$age),3)

sleep.mean <- round(mean(data$lfstl_sle),3)
sleep.sd <- round(sd(data$lfstl_sle),3)
sleep.iqr <- round(IQR(data$lfstl_sle),3)
sleep.skewness <- round(skewness(data$lfstl_sle),3)
sleep.sum <- round(sum(data$lfstl_sle),3)

inc.mean <- round(mean(data$family_income),3)
inc.sd <- round(sd(data$family_income),3)
inc.iqr <- round(IQR(data$family_income),3)
inc.skewness <- round(skewness(data$family_income),3)
inc.sum <- round(sum(data$family_income),3)

#binding values together in vectors
variable <- c("mean", "sd", "iqr", "skewness", "sum")
age <- c(age.mean, age.sd, age.iqr, age.skewness, age.sum)
sleep <- c(sleep.mean, sleep.sd, sleep.iqr, sleep.skewness, sleep.sum)
family_income <- c(inc.mean, inc.sd, inc.iqr, inc.skewness, inc.sum)

#binding vectors together to a table
measures.num <- rbind(variable, age, sleep, family_income)

#proportions, categories (for factors)
#summary table using table1
library(table1)

```

```
## Warning: package 'table1' was built under R version 4.2.3
```

```
##
## Attaching package: 'table1'
```

```
## The following objects are masked from 'package:Hmisc':
##
##   label, label<-, units
```

```
## The following objects are masked from 'package:base':
##
##   units, units<-
```

```
table1(~health_status+lfstl_vap+lfstl_smo+lfstl_alc+gender+education,data=data)
```

**Overall
(N=30328)**

health_status

poor	980 (3.2%)
------	------------

	Overall (N=30328)
fair	3275 (10.8%)
good	8467 (27.9%)
very_good	10627 (35.0%)
excellent	6979 (23.0%)
lfstl_vap	
yes	4393 (14.5%)
no	25935 (85.5%)
lfstl_smo	
yes	11485 (37.9%)
no	18843 (62.1%)
lfstl_alc	
yes	27084 (89.3%)
no	3244 (10.7%)
gender	
male	13954 (46.0%)
female	16374 (54.0%)
education	
primary_or_below	1925 (6.3%)
secondary	12387 (40.8%)
post_secondary	3984 (13.1%)
tertiary	12032 (39.7%)

To get a more intuitive understanding of the distribution of our data, it is useful to visually explore the data. There are several ways to go forward here:

For the metric variable (family_income), univariate distributions (plots 1-6):

- Histograms: they show the frequency distribution of a variable by dividing it into bins. They help visualize the shape, spread, and skewness of the data.
- Density plots: they show the probability density function of a variable, smoothing the distribution to highlight the overall shape and peaks. Contrary to histograms, they show a continuous estimate.
- Boxplots: they show the median, quartiles, and potential outliers of a variable. They are ideal for comparing distributions across groups.

For metric variables, bivariate distributions (plots 7-12):

- Histograms, density plots, and boxplots can also be help visualize bivariate (or multivariate) distributions. We plot family_income by gender.

For categorical variables, univariate distributions (plots 13-14):

- Bar plots: bar plots (or bar charts) look similar to histograms, but they visualize the number of cases per category. In this case, we visualize the distribution of the variable health_status.

For categorical variables, bivariate distributions (plots 15-16):

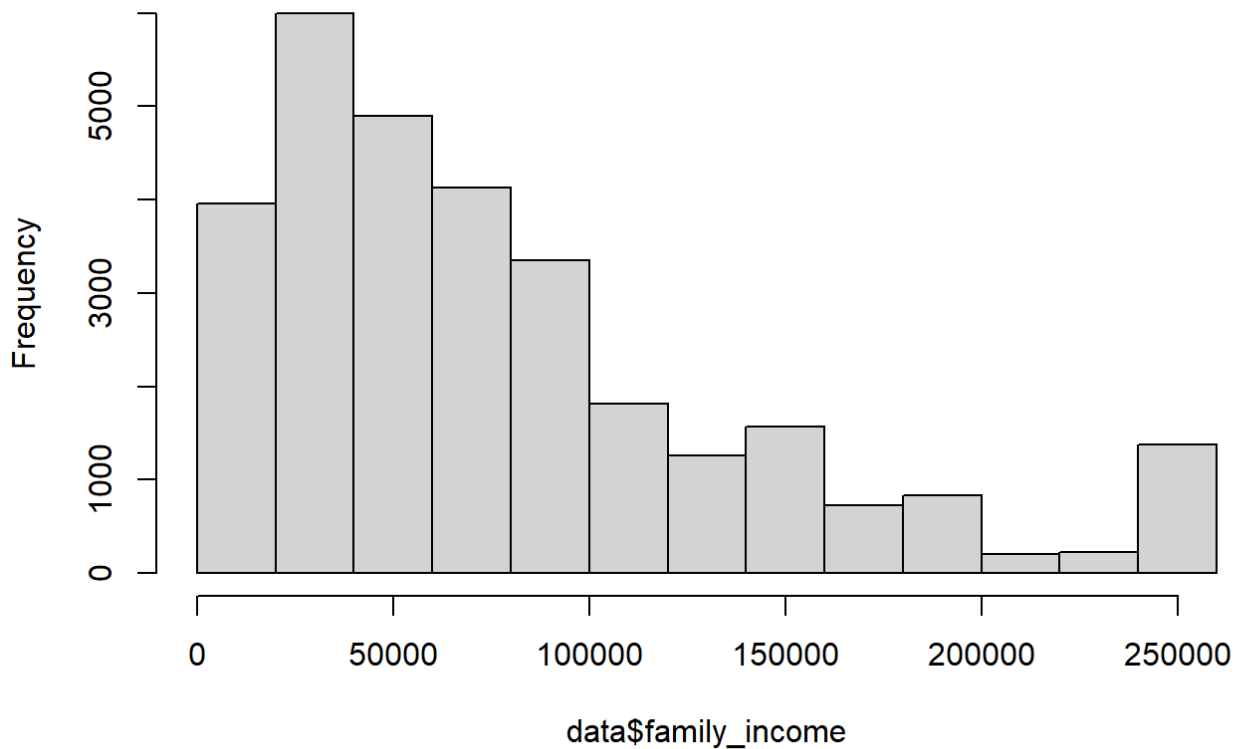
- Bar plots can also be used to show bivariate distributions. We visualize health_status by gender.

There is no clear answer to which one you should use, as all of them have their pro's and con's. However, think about who your audience is and what you want to demonstrate with the plot. Usually a plot is good, when the plot is informative and the main message can be seen very quickly, when the plot aesthetics are simple and not distracting, when there is no overplotting, when the plot elements are not confusing, and when the colors allow you to clearly distinguish the plot elements.

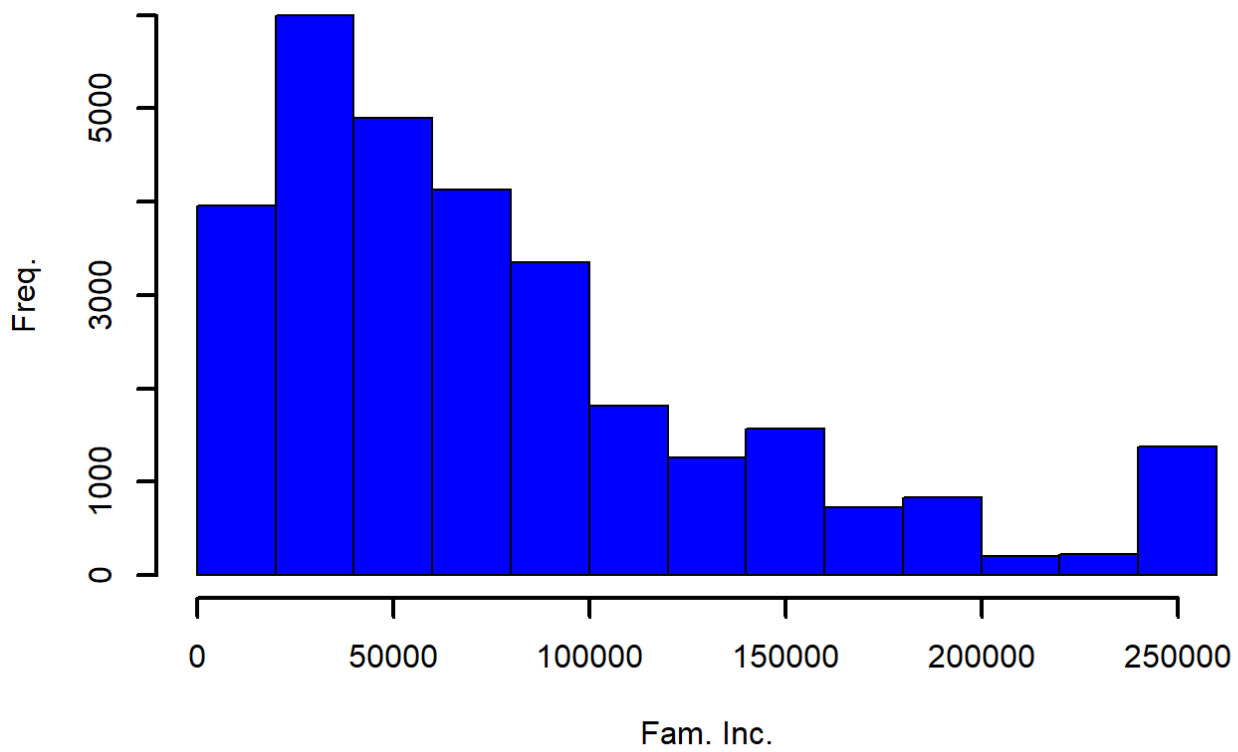
```
library(ggplot2)

#1: histogram (Base R)
plot(hist(data$family_income),
      main = "Histogram", xlab = "Fam. Inc.", ylab = "Freq.", col = "blue", lwd=2)
```

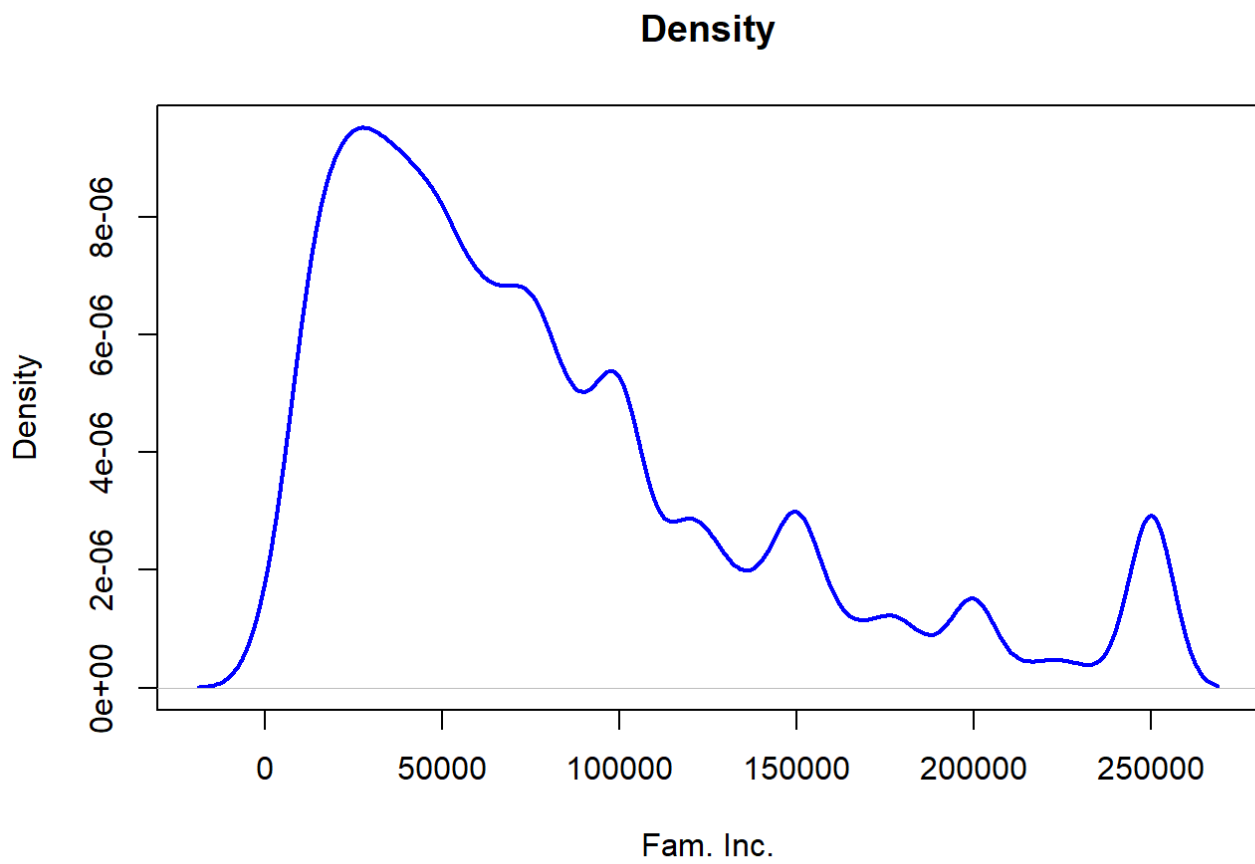
Histogram of data\$family_income



Histogram

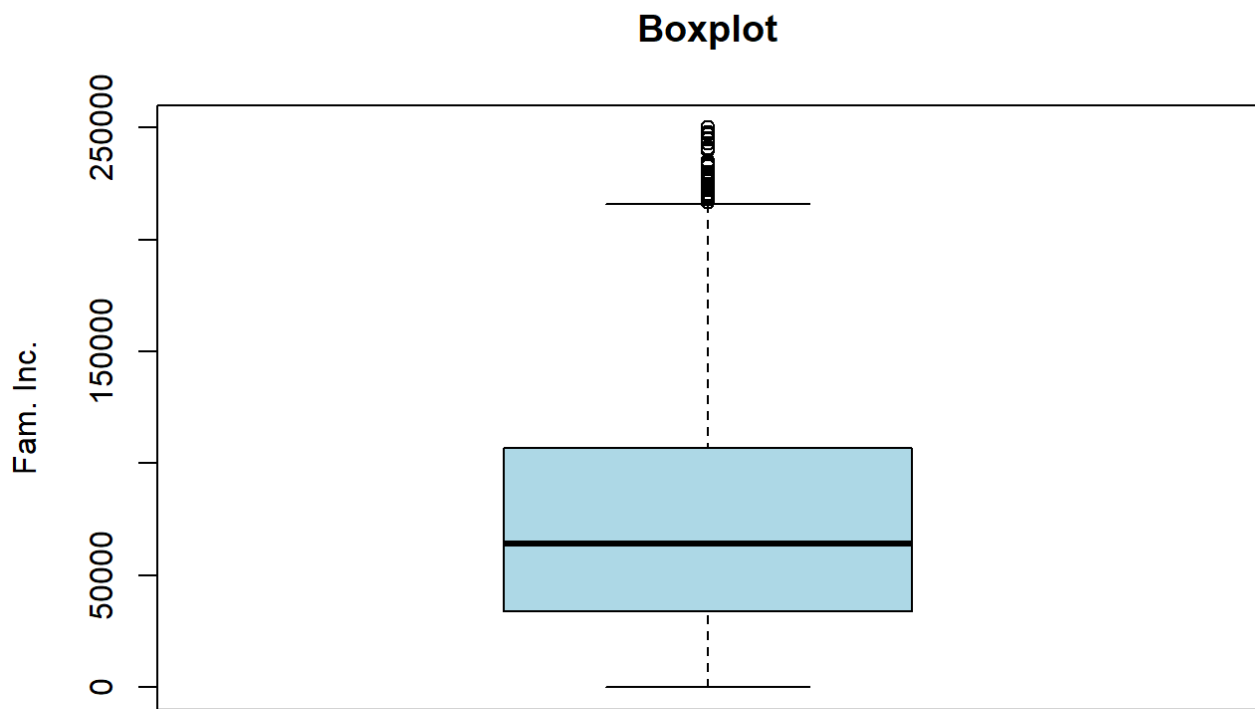


```
#2: density plot (Base R)
plot(density(data$family_income),
     main = "Density", xlab = "Fam. Inc.", ylab = "Density", col = "blue", lwd=2)
```

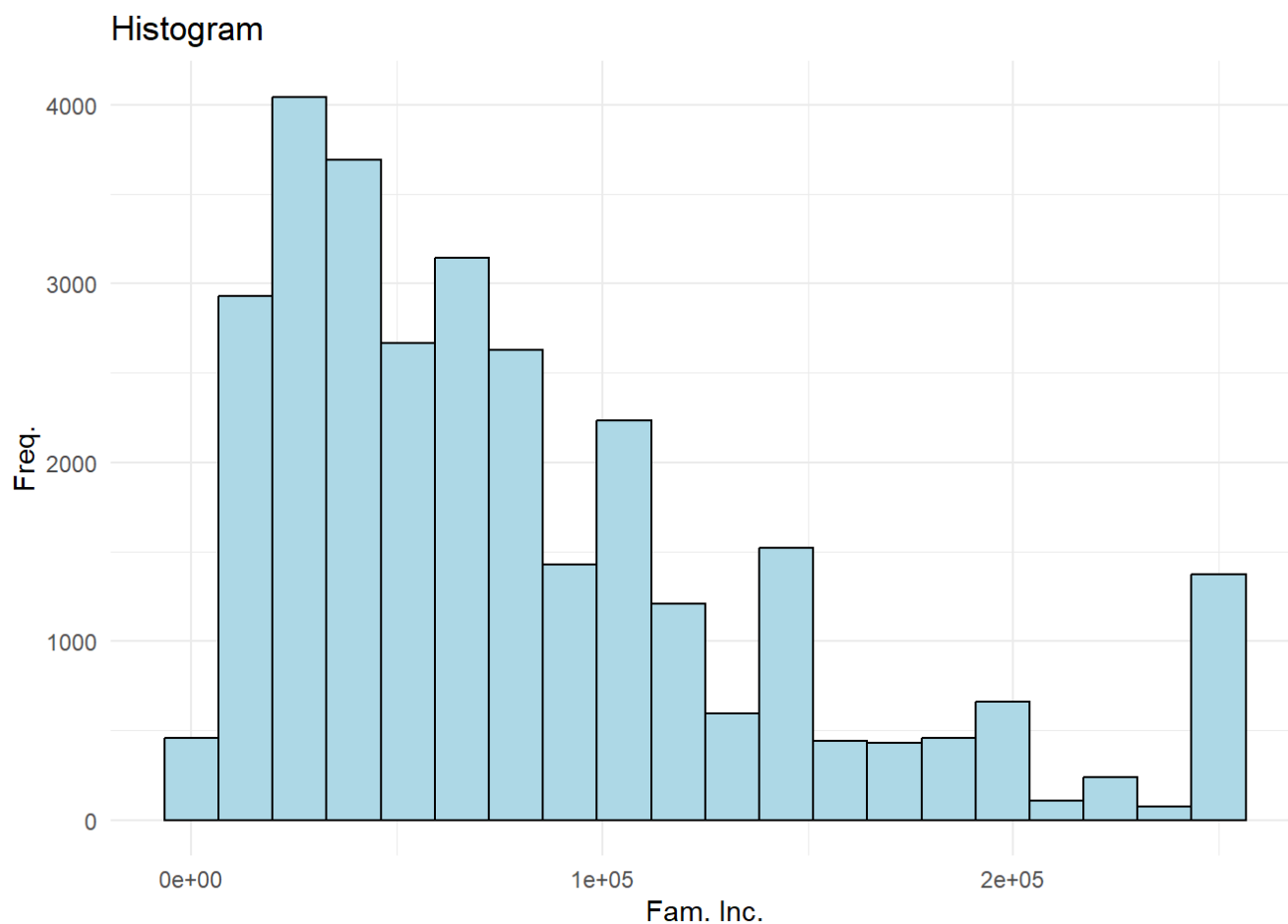


```
#3: boxplot (Base R)
```

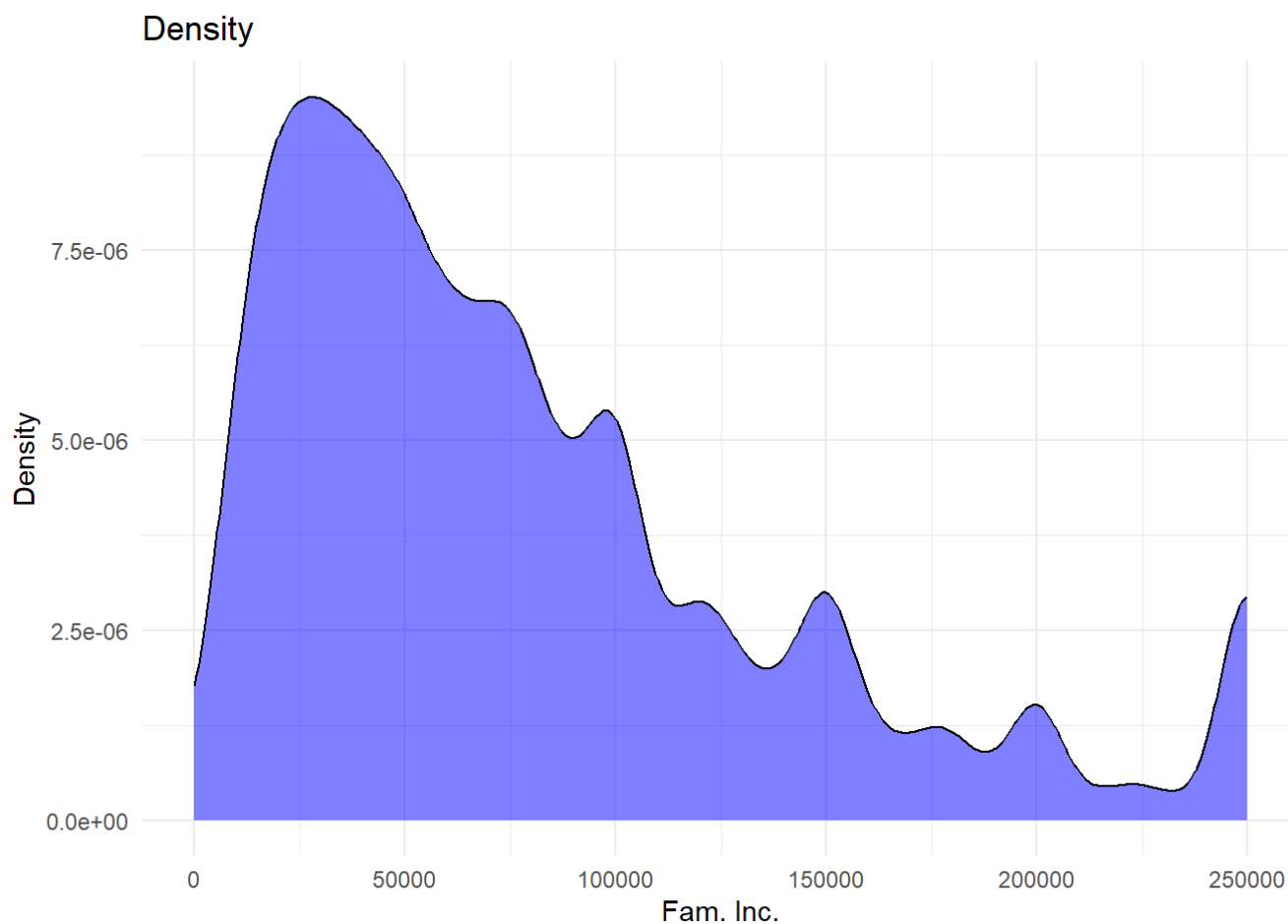
```
boxplot(data$family_income,  
        main = "Boxplot", ylab = "Fam. Inc.", col = "lightblue",border = "black")
```



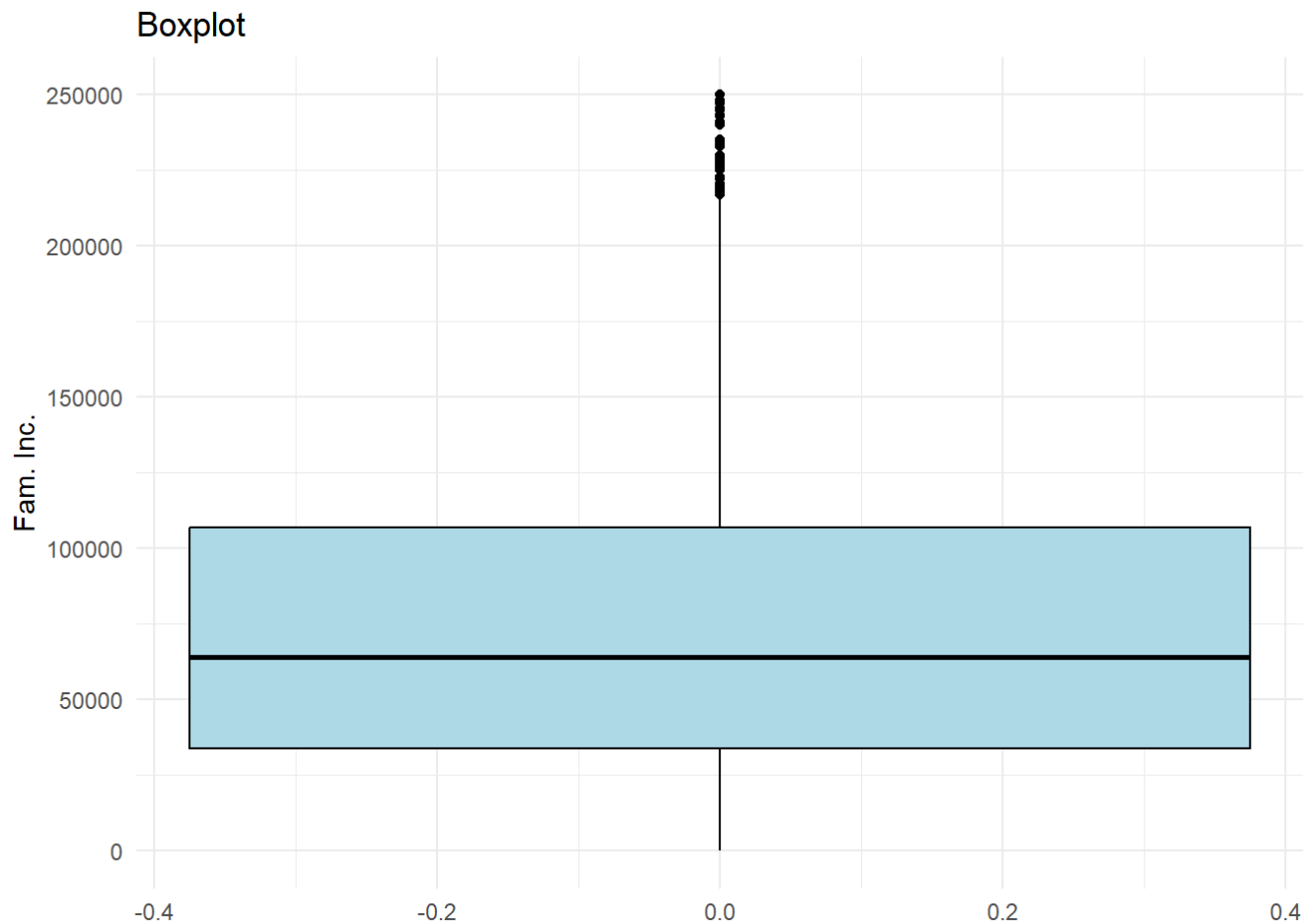
```
#4: histogram (ggplot2)
ggplot(data, aes(x = family_income)) +
  geom_histogram(bins = 20, fill = "lightblue", color = "black") +
  labs(title = "Histogram", x = "Fam. Inc.", y = "Freq.") +
  theme_minimal()
```



```
#5: density plot (ggplot2)
ggplot(data, aes(x = family_income)) +
  geom_density(fill = "blue", alpha = 0.5) +
  labs(title = "Density", x = "Fam. Inc.", y = "Density") +
  theme_minimal()
```



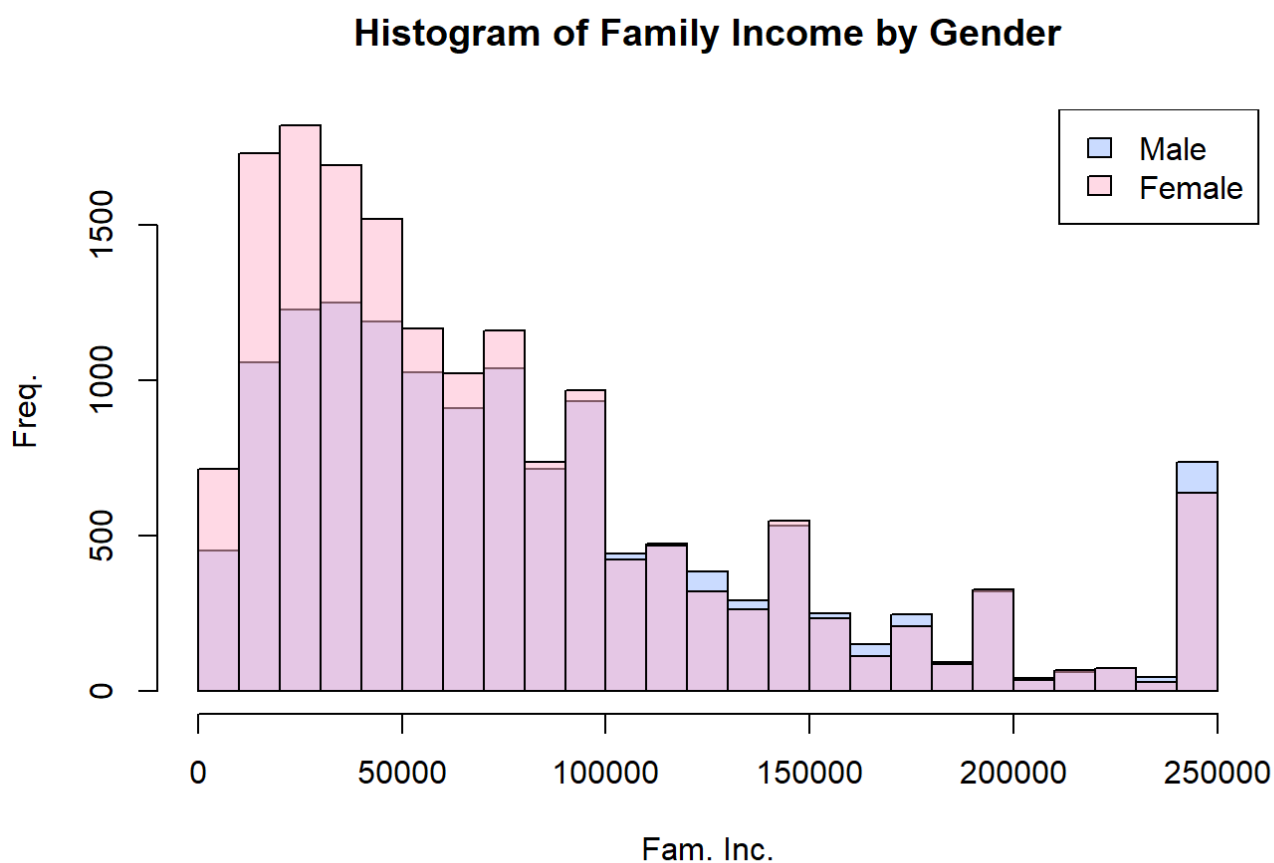
```
#6: boxplot (ggplot2)
ggplot(data, aes(y = family_income)) +
  geom_boxplot(fill = "lightblue", color = "black") + labs(title = "Boxplot", y = "Fam. In
c.") +
  theme_minimal()
```



```
#####
#separated by gender
#####

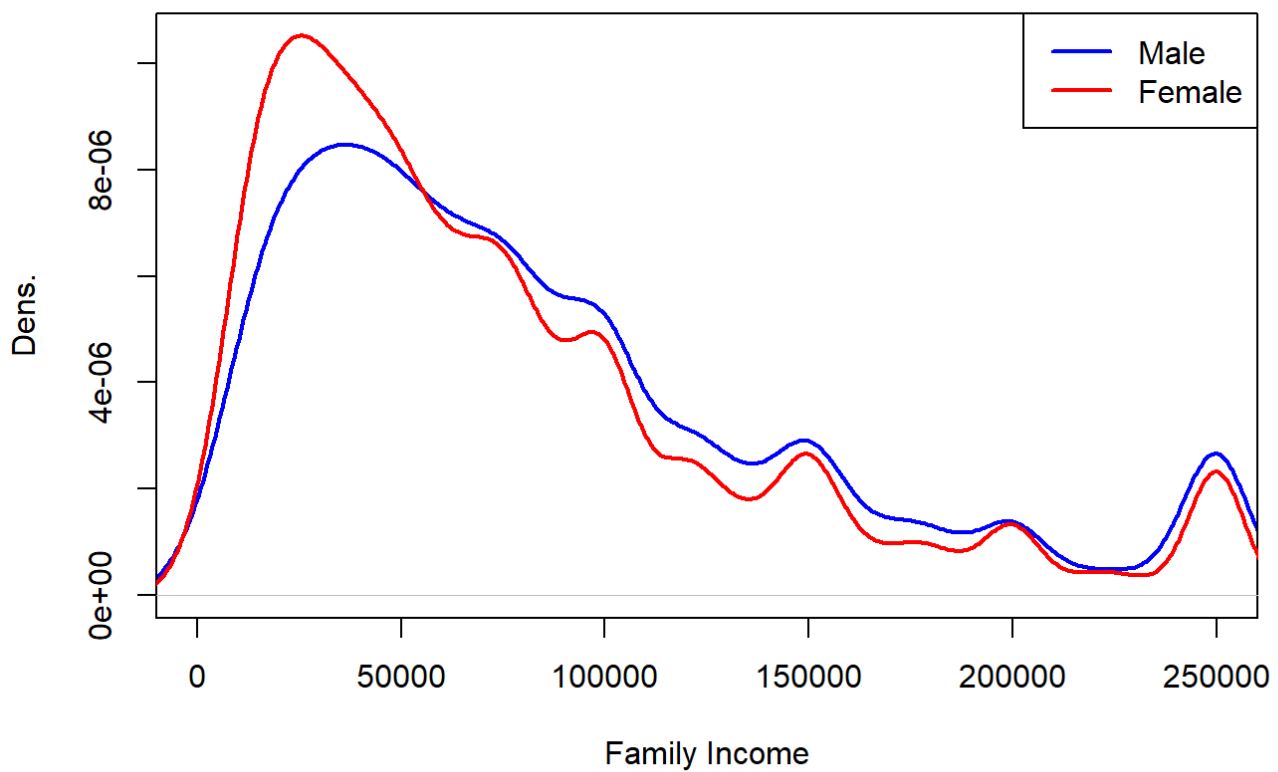
#7: histogram by gender (Base R)
male_income <- data$family_income[data$gender == "male"]
female_income <- data$family_income[data$gender == "female"]

hist(male_income,
     main = "Histogram of Family Income by Gender",
     xlab = "Fam. Inc.",
     ylab = "Freq.",
     col = rgb(0.7, 0.8, 1, alpha = 0.7),
     xlim = range(c(male_income, female_income)),
     ylim = range(0, 1800),
     breaks = 20)
hist(female_income,
     col = rgb(1, 0.7, 0.8, alpha = 0.5),
     add = TRUE,
     breaks = 20)
legend("topright",
     legend = c("Male", "Female"),
     fill = c(rgb(0.7, 0.8, 1, alpha = 0.7), rgb(1, 0.7, 0.8, alpha = 0.5)),
     border = "black")
```

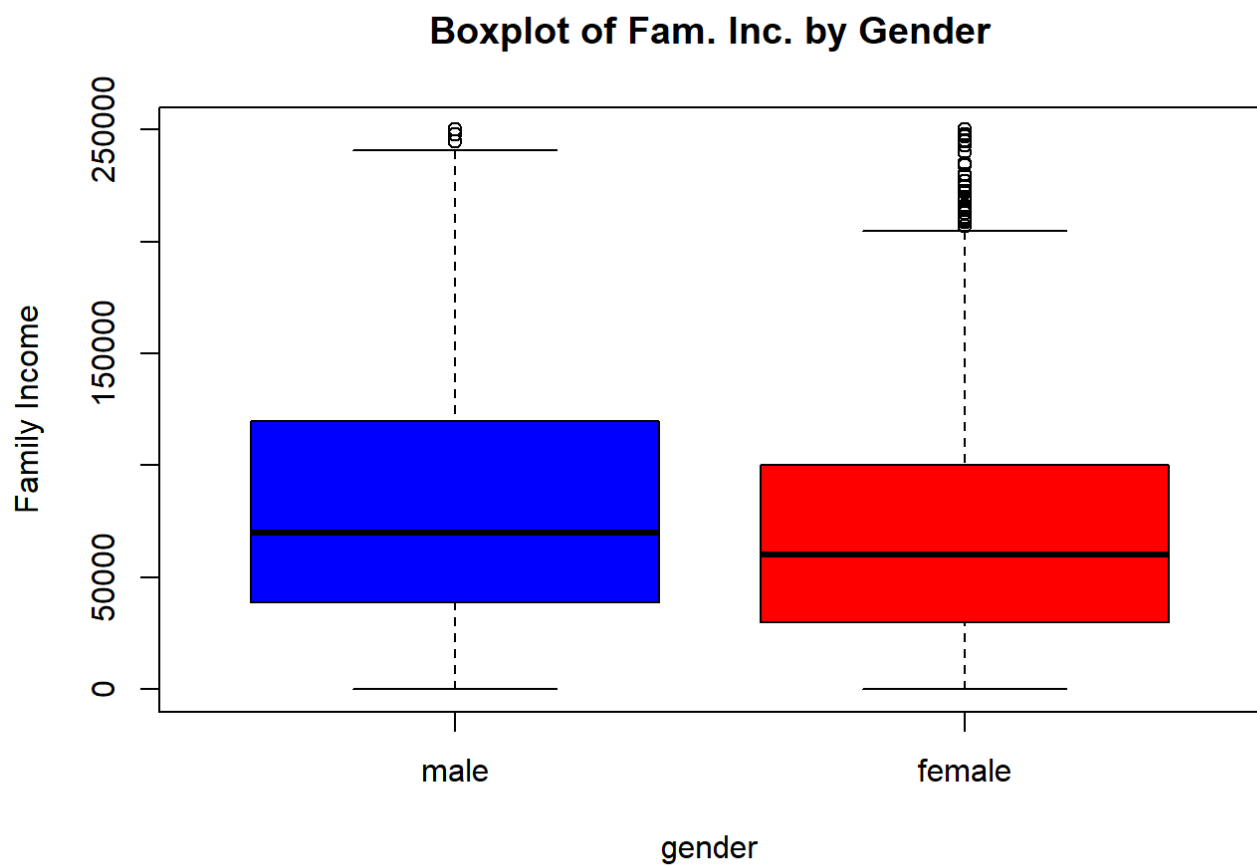


#8: density plot by gender (Base R)

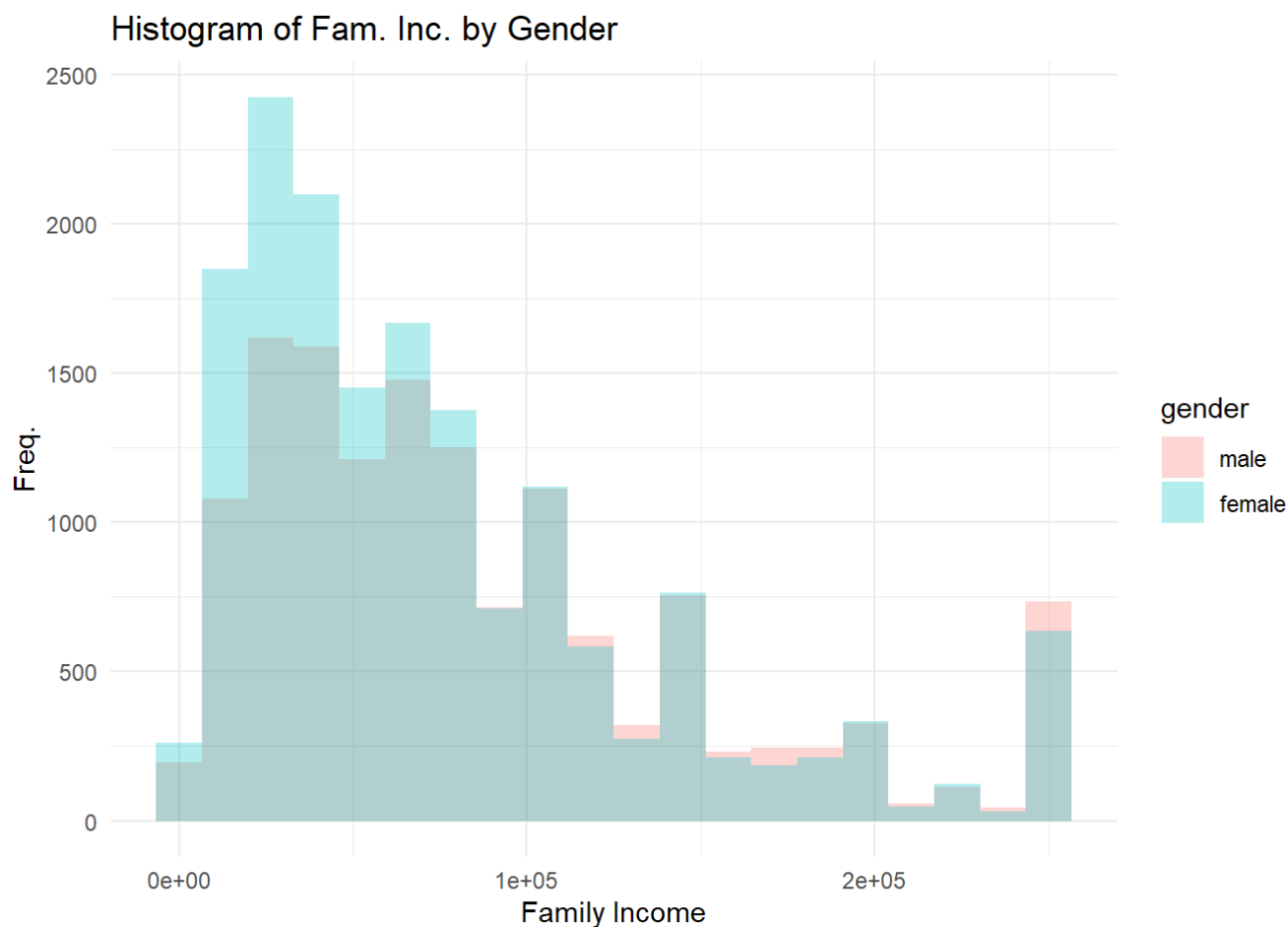
```
plot(density(male_income),
     main = "Density Plot of Fam. Inc. by Gender",
     xlab = "Family Income",
     ylab = "Dens.",
     col = "blue",
     lwd = 2,
     xlim = range(c(male_income, female_income)),
     ylim = c(0, max(density(male_income)$y, density(female_income)$y)))
lines(density(female_income), col = "red", lwd = 2)
legend("topright",
      legend = c("Male", "Female"),
      col = c("blue", "red"),
      lwd = 2)
```

Density Plot of Fam. Inc. by Gender

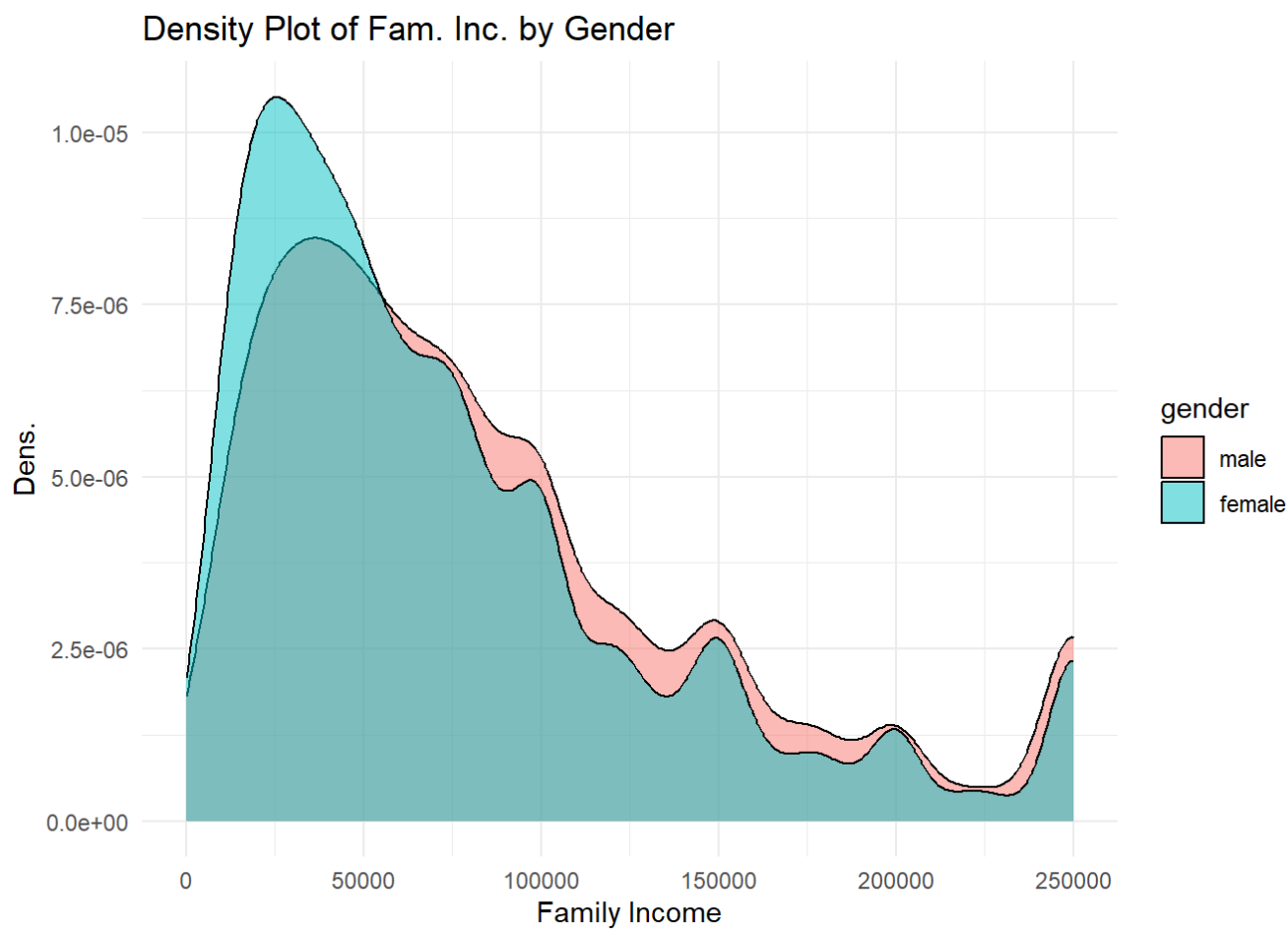
```
#9: boxplot by gender (Base R)
boxplot(family_income ~ gender,
        data = data,
        main = "Boxplot of Fam. Inc. by Gender",
        ylab = "Family Income",
        col = c("blue", "red"),
        border = "black")
```



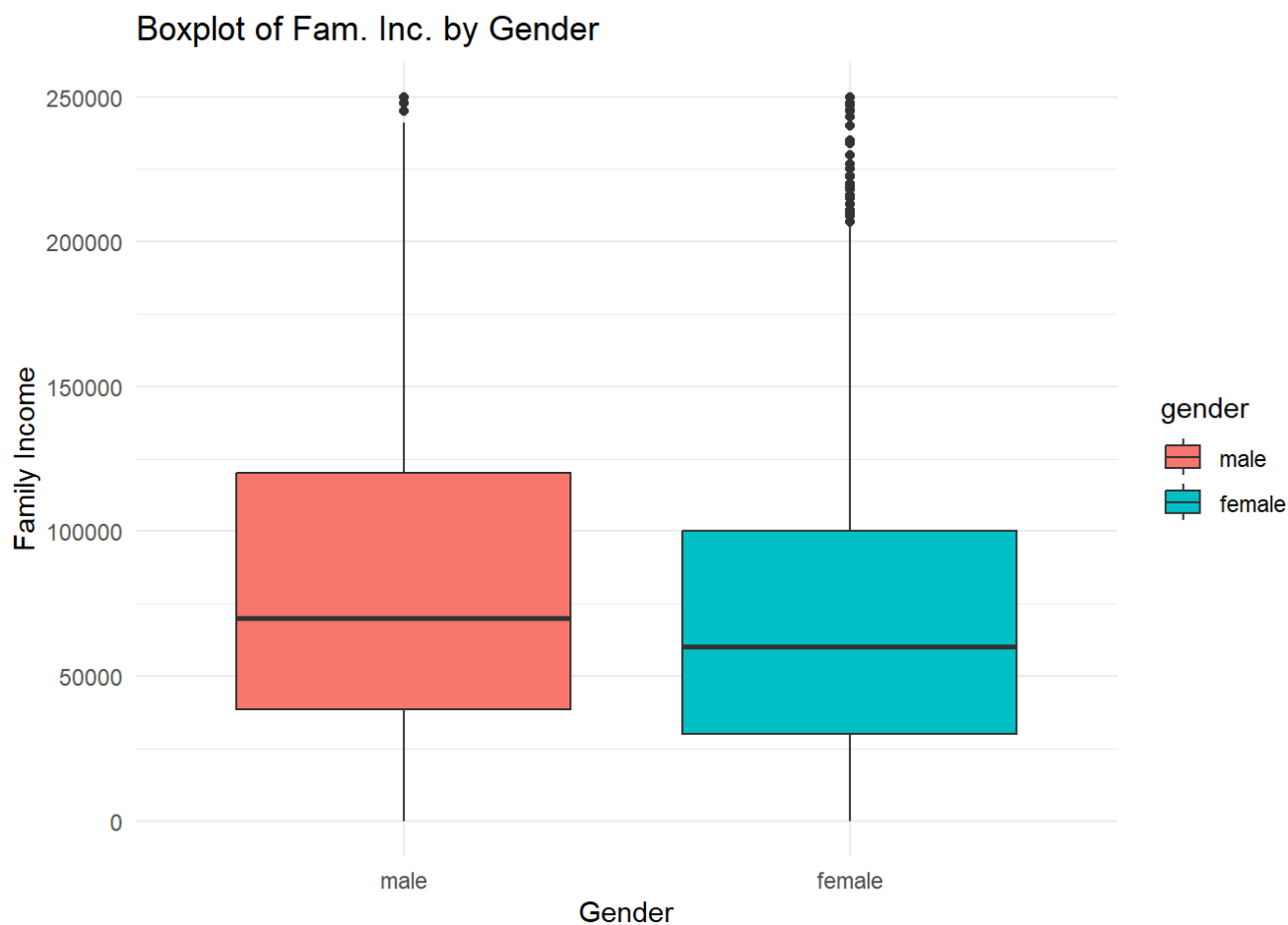
```
#10: histogram by gender (ggplot2)
ggplot(data, aes(x = family_income, fill = gender)) +
  geom_histogram(bins = 20, position = "identity", alpha = 0.3) +
  labs(title = "Histogram of Fam. Inc. by Gender",
        x = "Family Income",
        y = "Freq.") +
  theme_minimal()
```



```
#11: density plot by gender (ggplot2)
ggplot(data, aes(x = family_income, fill = gender)) +
  geom_density(alpha = 0.5) +
  labs(title = "Density Plot of Fam. Inc. by Gender",
       x = "Family Income",
       y = "Dens.") +
  theme_minimal()
```



```
#12: boxplot by gender (ggplot2)
ggplot(data, aes(x = gender, y = family_income, fill = gender)) +
  geom_boxplot() +
  labs(title = "Boxplot of Fam. Inc. by Gender",
       x = "Gender",
       y = "Family Income") +
  theme_minimal()
```

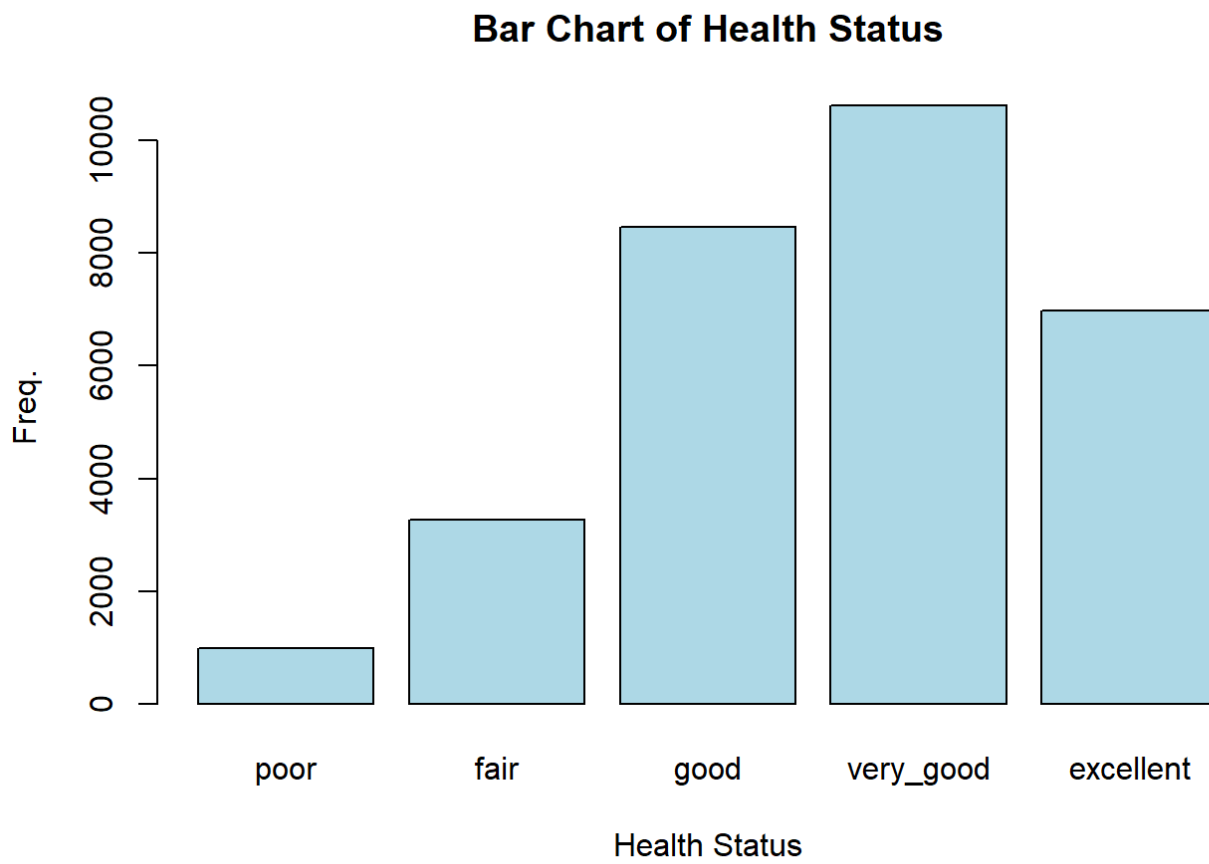


```
#####
```

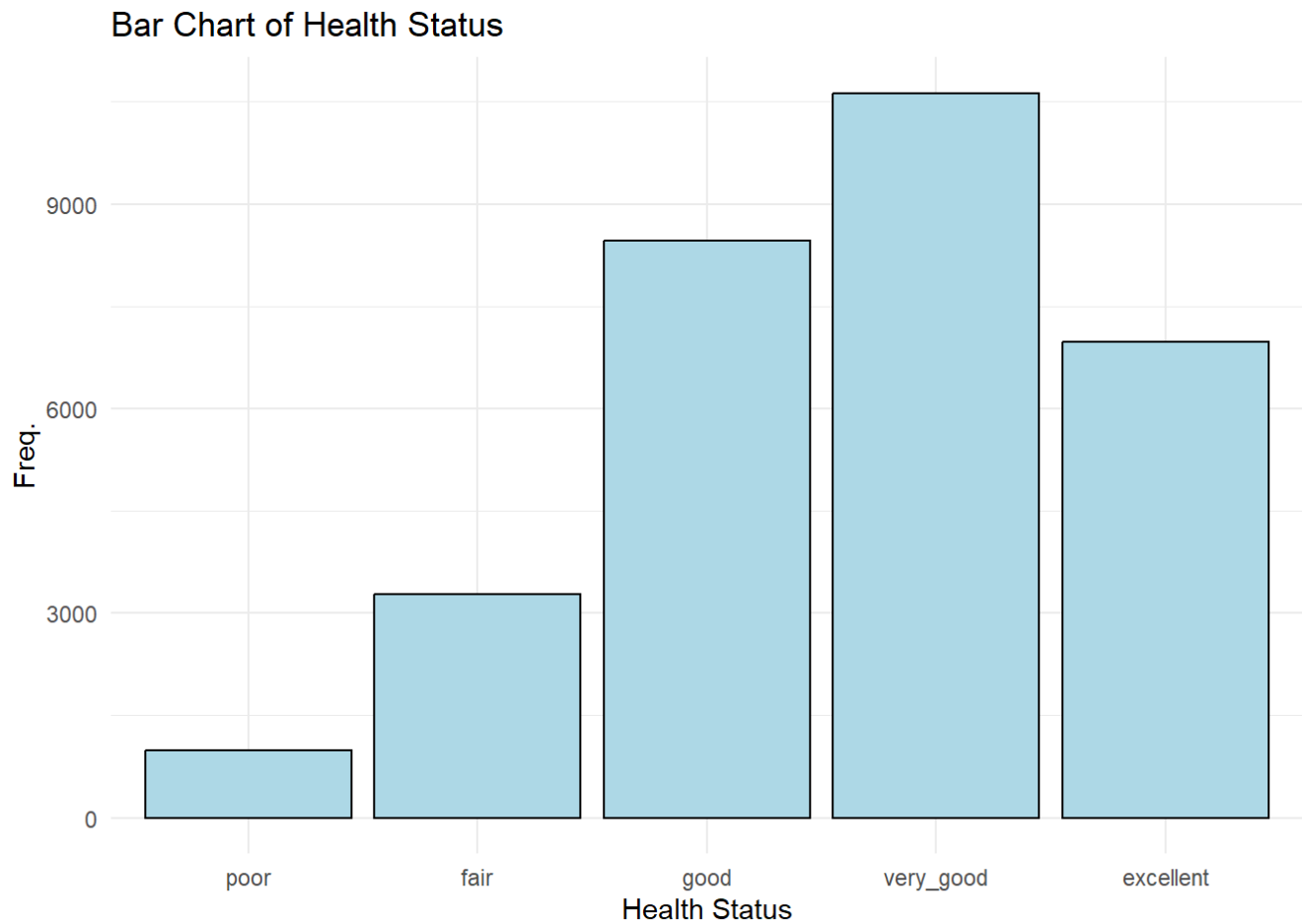
```
#bar charts for categorical variables
```

```
#13: health_status (Base R)
```

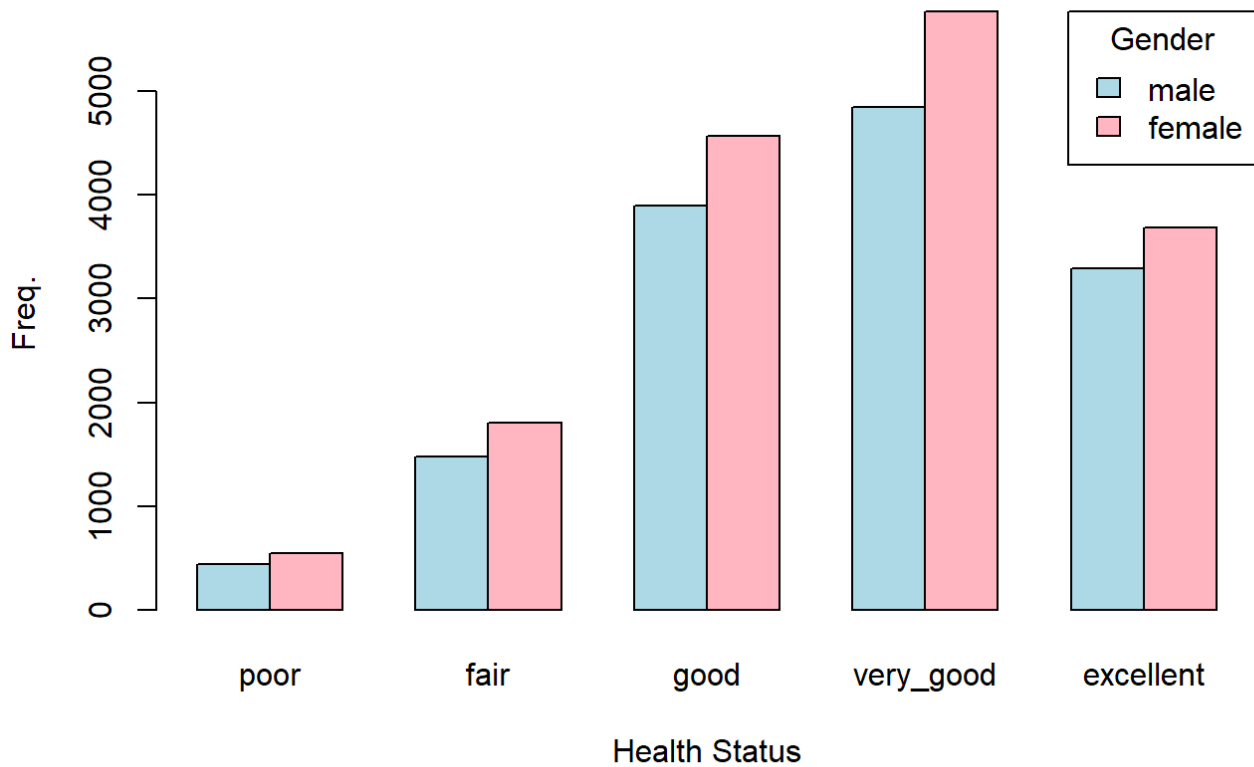
```
barplot(table(data$health_status),  
        main = "Bar Chart of Health Status",  
        xlab = "Health Status",  
        ylab = "Freq.",  
        col = "lightblue",  
        border = "black")
```



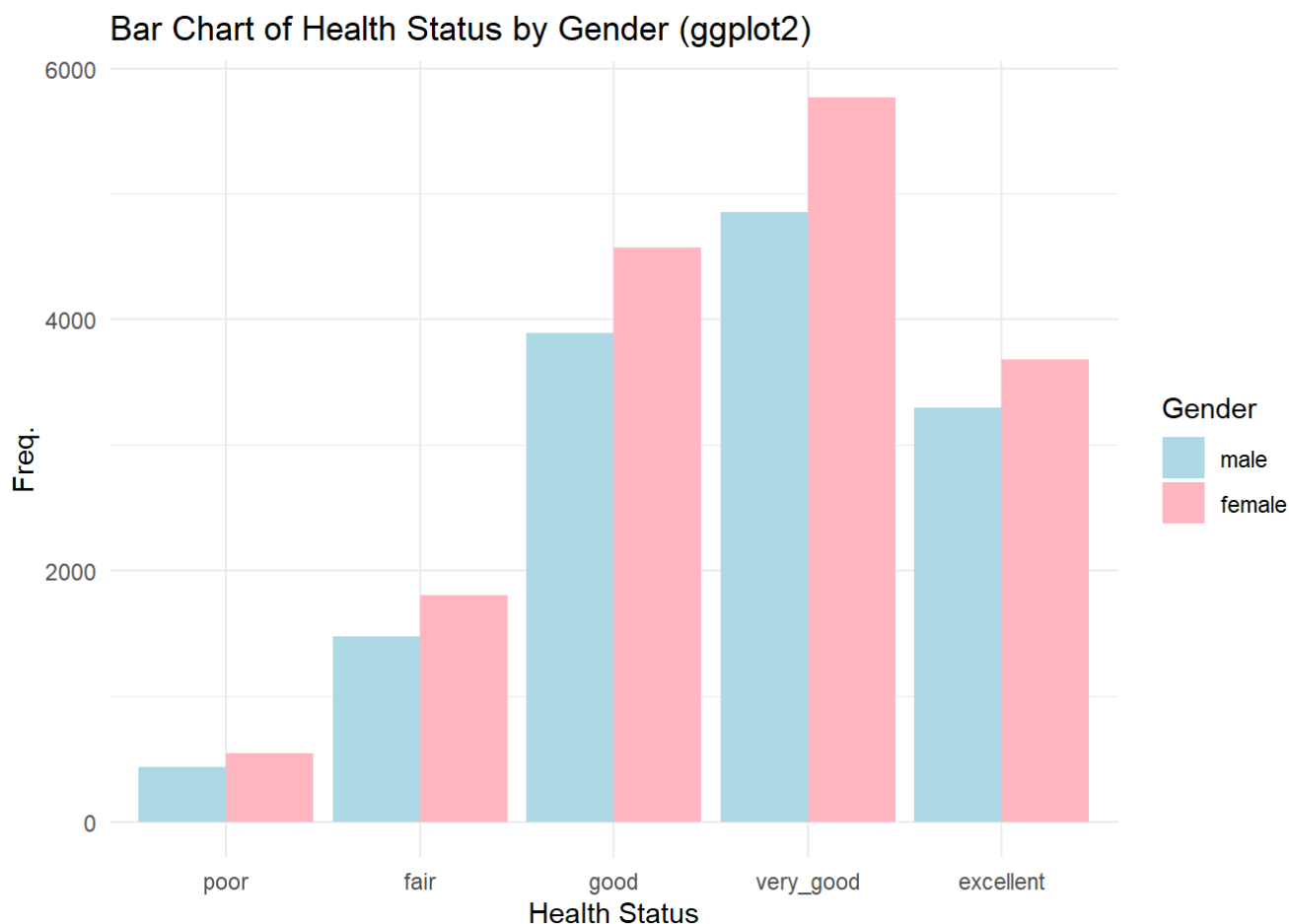
```
#14: health_status (ggplot2)
ggplot(data, aes(x = health_status)) +
  geom_bar(fill = "lightblue", color = "black") +
  labs(title = "Bar Chart of Health Status",
       x = "Health Status",
       y = "Freq.") +
  theme_minimal()
```



```
#####  
#separated by gender  
#####  
  
#15: health_status by gender (Base R)  
barplot(table(data$gender, data$health_status),  
        beside = TRUE,  
        main = "Bar Chart of Health Status by Gender",  
        xlab = "Health Status",  
        ylab = "Freq.",  
        col = c("lightblue", "lightpink"),  
        legend.text = rownames(table(data$gender, data$health_status)),  
        args.legend = list(title = "Gender", x = "topright", border = "black"),  
        names.arg = colnames(table(data$gender, data$health_status)))
```

Bar Chart of Health Status by Gender

```
#16: health_status by gender (ggplot2)
ggplot(data, aes(x = health_status, fill = gender)) +
  geom_bar(position = "dodge") +
  labs(title = "Bar Chart of Health Status by Gender (ggplot2)",
       x = "Health Status",
       y = "Freq.",
       fill = "Gender") +
  scale_fill_manual(values = c("lightblue", "lightpink")) +
  theme_minimal()
```



After plotting the distributions, we can go a step further and explore how strong some variables are correlated.

- 1: We can start by calculating some point estimates (mean, median etc.) again, this time by another variable. In the first example, we look at family_income by health_status.
- 2: The `cor()` function in R calculates the correlation coefficient (by default Pearson) between numeric variables. It shows the strength and direction of their linear relationship. We create a plot using age, income, health_status, and education. `cor()` only creates a numeric output, we can use, for example, `corrplot()` or `ggcorrplot()` to visualize the correlations.
- 3: After investigating the correlation, it's useful to look at the distribution of the individual data points, with one variable on the x-axis, and the other on the y-axis. This helps to detect patterns and clusters, and outliers.
- 4: While scatterplots are a popular choice, they sometimes have an overplotting problem, meaning that too many data points are at the same position. This is especially the case, when the range of the variables is small. One alternative is to use jitterplots instead of scatterplots. Another alternative approach is the use of heatmaps, which show the concentration of data points in smaller tiles.

```
#example health_status & family_income
```

```
#1: grouped summaries
```

```
data_grouped <- data %>%  
  group_by(health_status) %>%  
  summarise(  
    mean_fam_inc = mean(family_income),  
    median_fam_inc = median(family_income),  
    max_fam_inc = max(family_income),  
    min_fam_inc = min(family_income),  
    count = n()  
  )
```

```
#2: correlation table
```

```
data2 <- data[c("age", "family_income", "health_status", "education")]
```

```
#the cor() function is for numeric variables, we therefore transform the levels of the two  
factors health_status and education into numbers
```

```
data2$health_status <- as.numeric(data2$health_status)
```

```
data2$education <- as.numeric(data2$education)
```

```
corr.table <- cor(data2, use = "complete.obs")
```

```
#2: corrplot
```

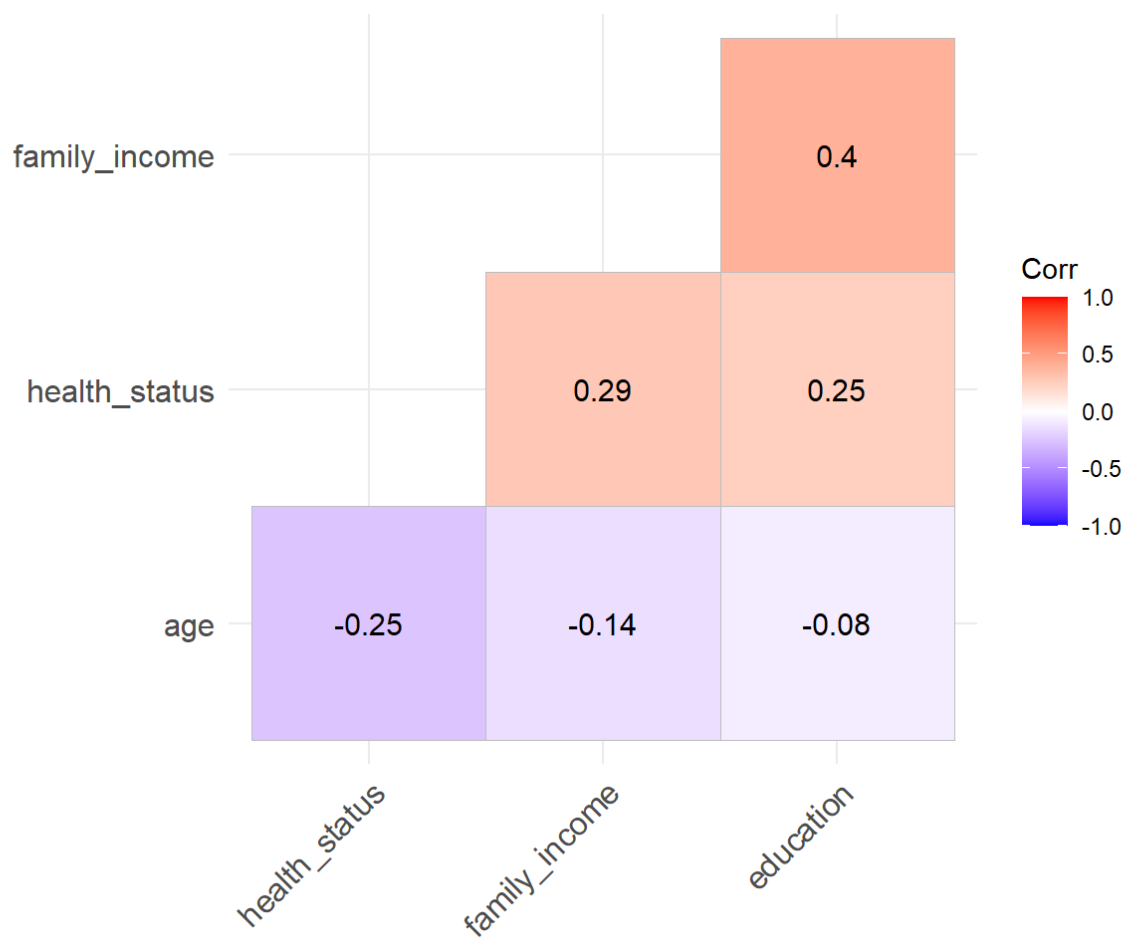
```
library(corrplot)
```

```
## corrplot 0.92 loaded
```

```
corrplot(corr.table, method="number")
```

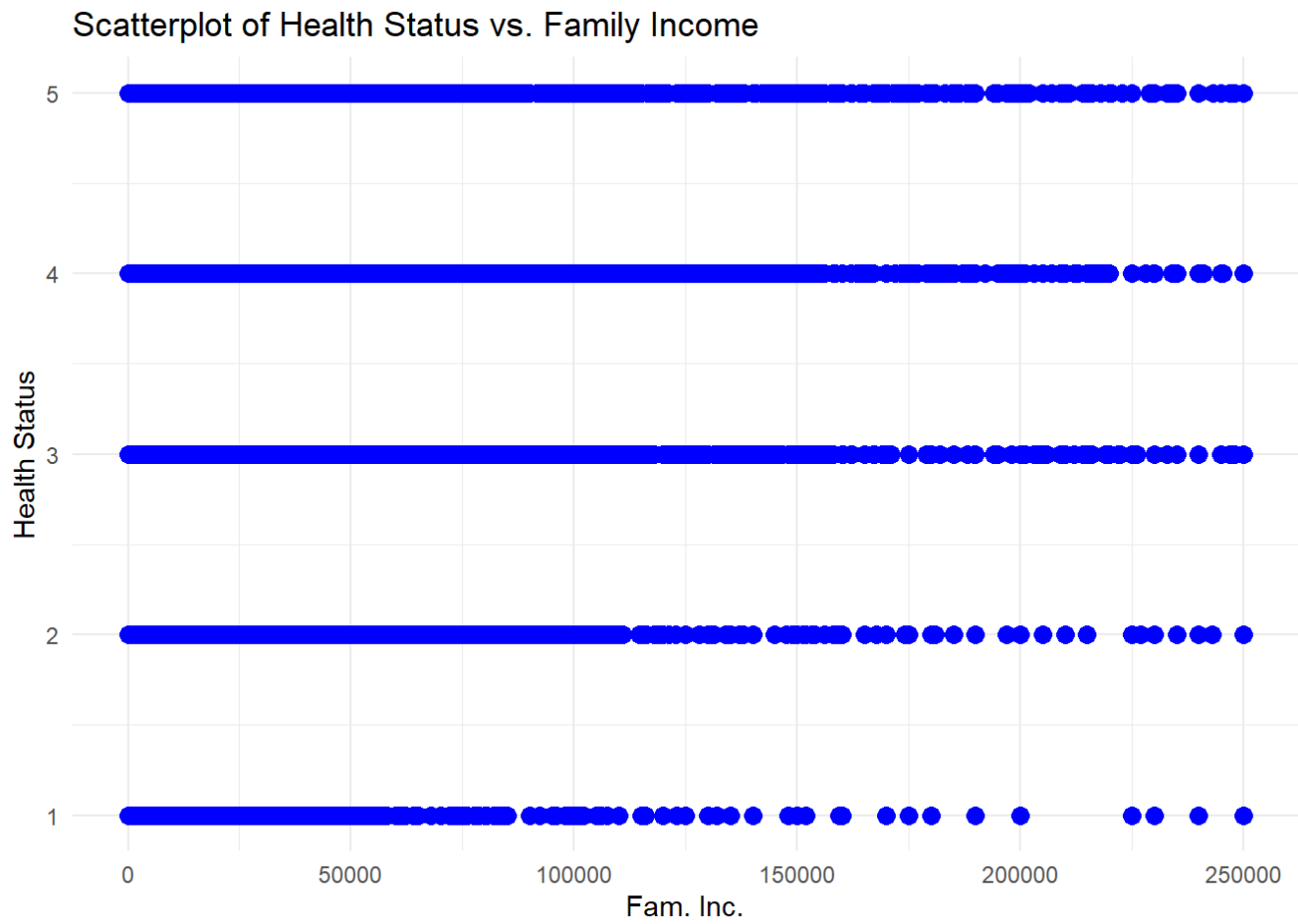


```
library(ggcorrplot)
ggcorrplot(corr.table,method = "square", type = "lower",lab=TRUE,hc.order=TRUE)
```

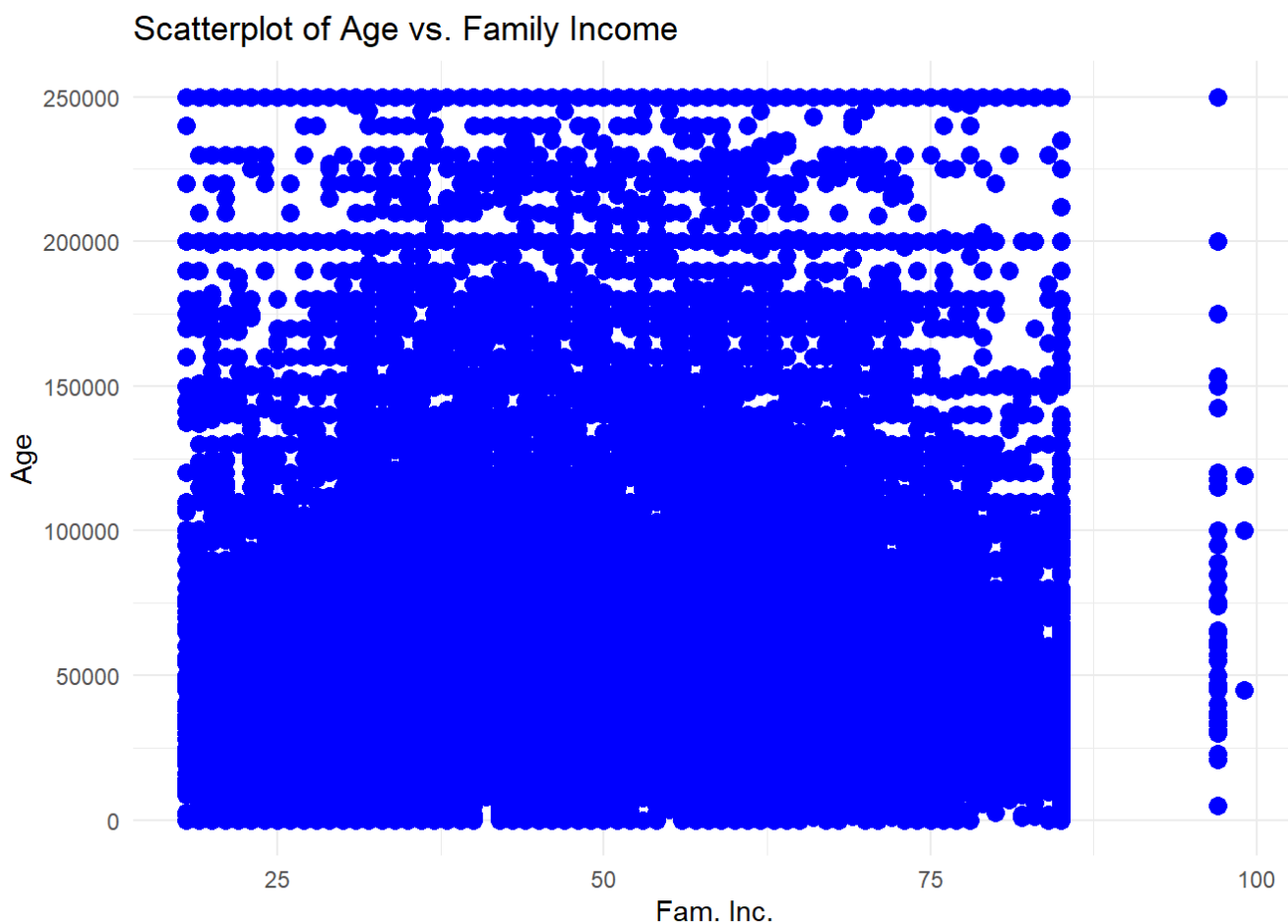


#3: scatterplot

```
ggplot(data, aes(x = family_income, y = as.numeric(health_status))) +  
  geom_point(color = "blue", size = 3) +  
  labs(title = "Scatterplot of Health Status vs. Family Income",  
        x = "Fam. Inc.",  
        y = "Health Status") +  
  theme_minimal()
```

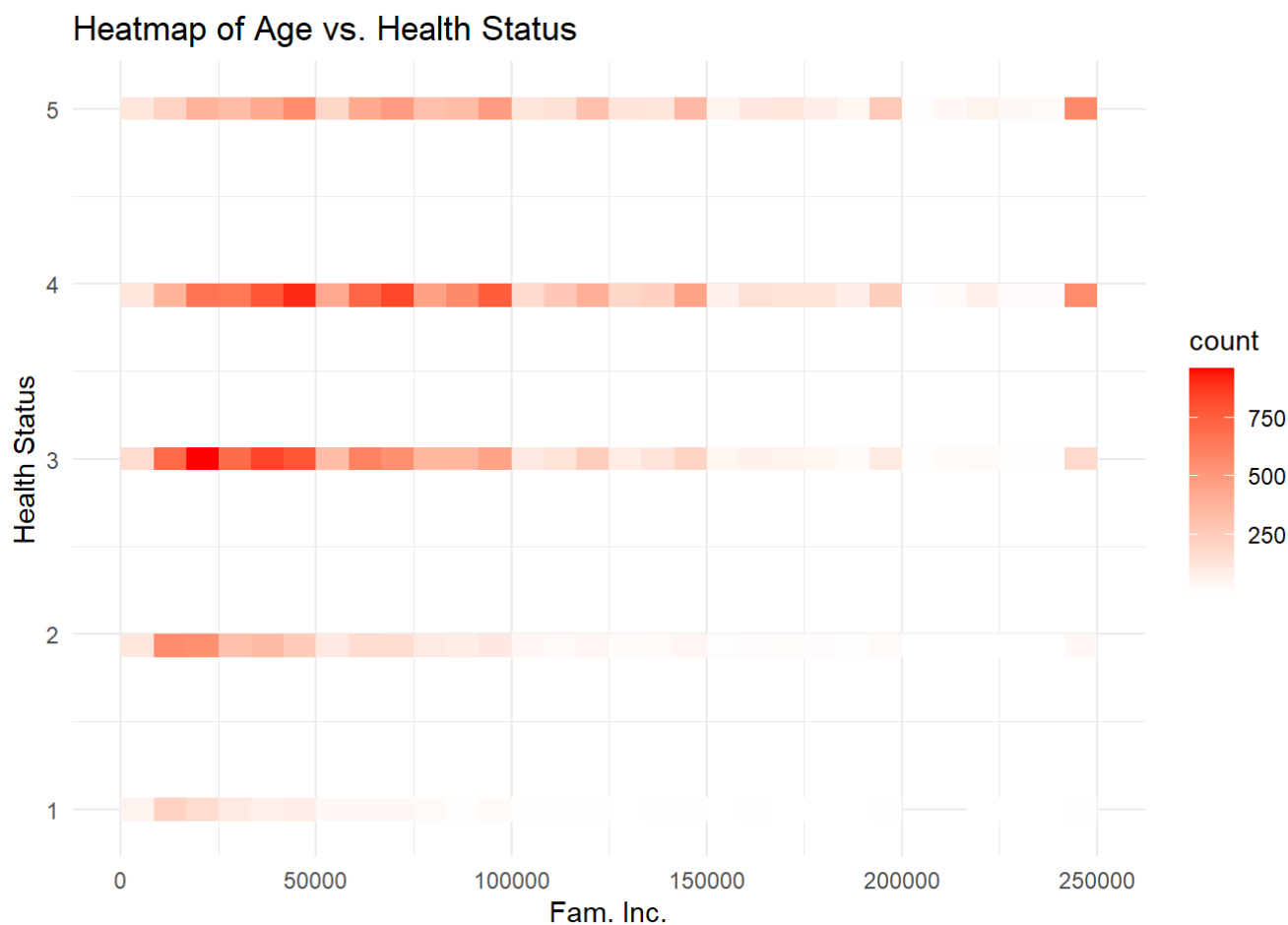


```
ggplot(data, aes(x = age, y = family_income)) +  
  geom_point(color = "blue", size = 3) +  
  labs(title = "Scatterplot of Age vs. Family Income",  
        x = "Fam. Inc.",  
        y = "Age") +  
  theme_minimal()
```

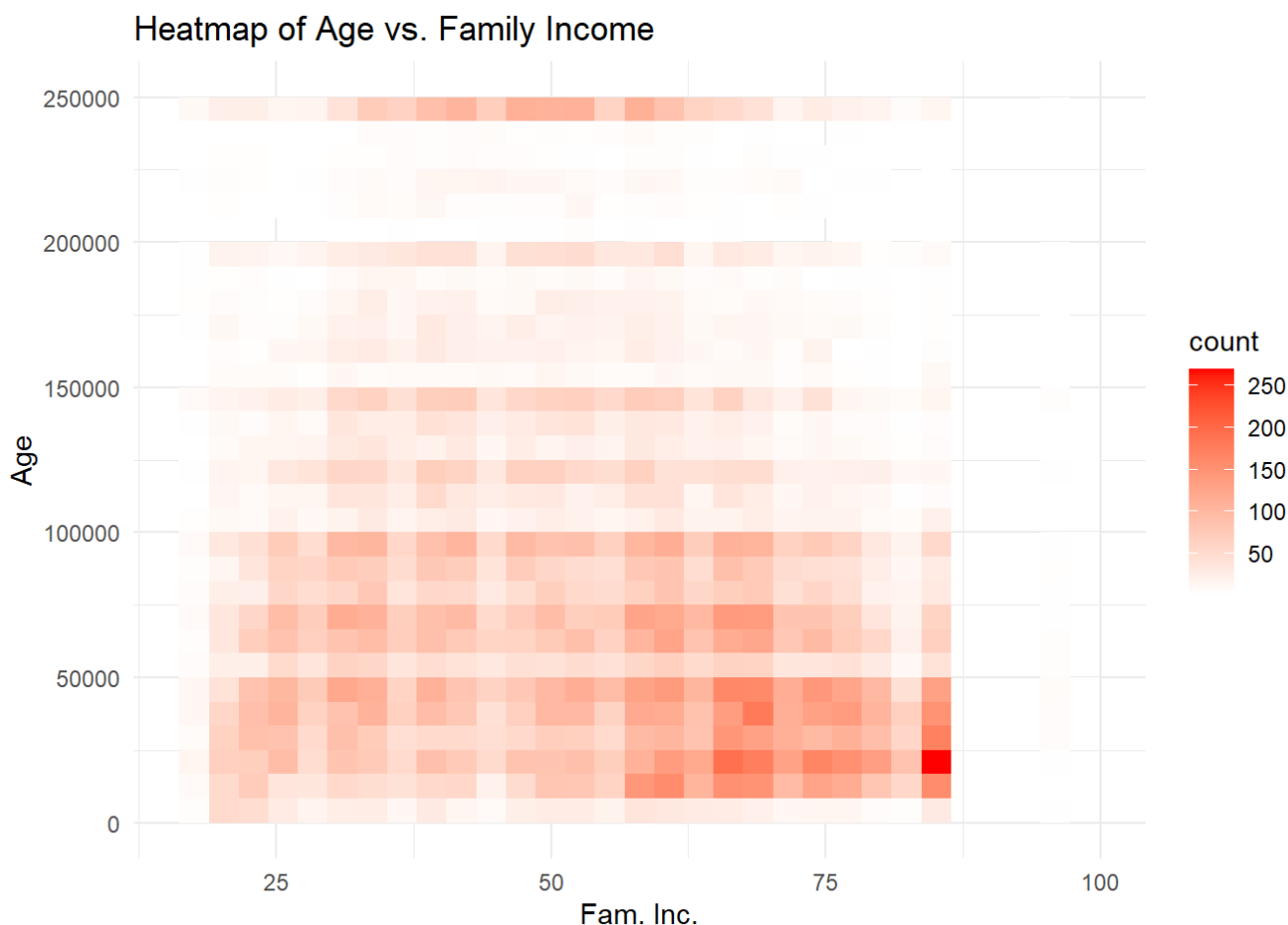


#4: heatmap

```
ggplot(data, aes(x = family_income, y = as.numeric(health_status))) +  
  geom_bin2d() +  
  scale_fill_gradient(low = "white", high = "red") +  
  labs(title = "Heatmap of Age vs. Health Status", x = "Fam. Inc.", y = "Health Status") +  
  theme_minimal()
```



```
ggplot(data, aes(x = age, y = family_income)) +  
  geom_bin2d() +  
  scale_fill_gradient(low = "white", high = "red") +  
  labs(title = "Heatmap of Age vs. Family Income", x = "Fam. Inc.", y = "Age") +  
  theme_minimal()
```



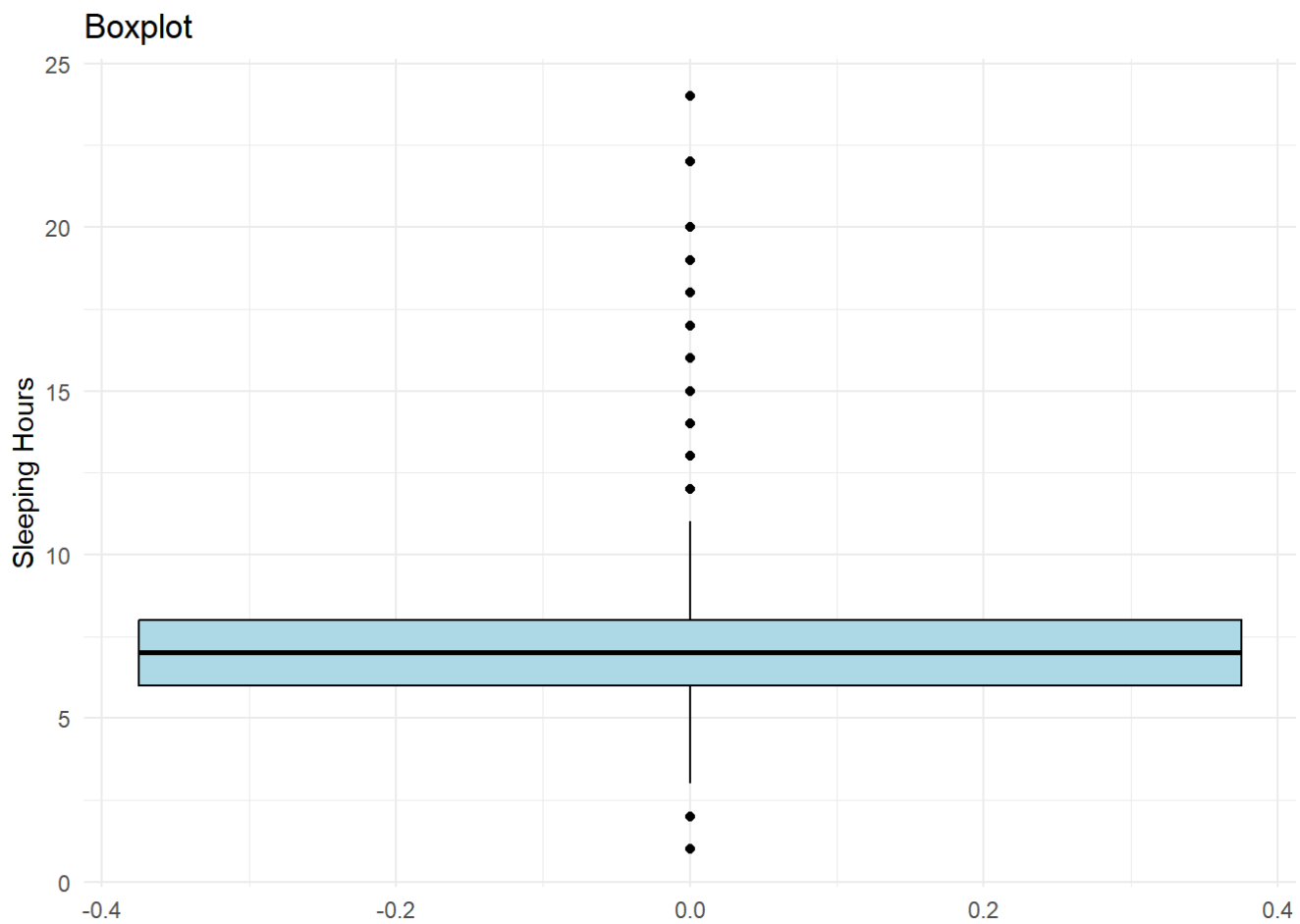
We should also assume that there might be outliers in our data that can distort the main message of our analysis. There are different techniques to recognize outliers. The decision of what to do with outliers, should be given careful consideration. Whether we decide to remove them, to keep them, to recode them, or any other treatment, we should not forget that they are also legitimate observations. We should therefore always be fully transparent about what and why we do something to them, and what consequences this could have.

A common way is also to do further inferential analysis once with a full dataset, and once with an outlier-free dataset.

- 0: a visual check, for example, with a boxplot can help to see, if outliers are present.
- 1: One way would be to remove outliers if their z-scores (after standardizing them) exceeds a certain threshold.
- 2: a common rule is the one that is also implemented in the boxplots. Outliers are those data point, that exceed the IQR multiplied by 1.5 in either direction.
- 3: another way would be to remove them by percentile, for example, by removing the top 5 and the bottom 5 percent of a variable (not recommended).
- 4: we can come up with our own rules, either theoretically or methodologically backed. In the fourth example, we exclude outliers based on a rule for which we used median and median absolute deviation instead of mean and standard deviation.

```
#0: visual check for outliers
```

```
ggplot(data, aes(y = lfstl_sle)) +  
  geom_boxplot(fill = "lightblue", color = "black") + labs(title = "Boxplot", y = "Sleepin  
g Hours") +  
  theme_minimal()
```



```

#1: z-scores
data$lfstl_sle_outliers1 <- abs(scale(data$lfstl_sle)) > 2

#2: IQR range
Q1 <- quantile(data$lfstl_sle, 0.25)
Q3 <- quantile(data$lfstl_sle, 0.75)
IQR <- Q3 - Q1
lower_bound <- Q1 - 1.5 * IQR
upper_bound <- Q3 + 1.5 * IQR

data$lfstl_sle_outliers2 <- ifelse(data$lfstl_sle < lower_bound | data$lfstl_sle > upper_bound, TRUE, FALSE)

#3: percentiles
lower_percentile <- quantile(data$lfstl_sle, 0.05)
upper_percentile <- quantile(data$lfstl_sle, 0.95)
data$lfstl_sle_outliers3 <- ifelse(data$lfstl_sle < lower_percentile | data$lfstl_sle > upper_percentile, TRUE, FALSE)

#4: modified z-scores
median <- median(data$lfstl_sle)
mad <- mad(data$lfstl_sle)
modified_z <- 0.6745 * (data$lfstl_sle - median) / mad
data$lfstl_sle_outliers4 <- abs(modified_z) > 3.5

#follow-up: analysing outliers
outlier_data <- data %>%
  filter(lfstl_sle_outliers1 == TRUE)

```

```

## Warning: Using one column matrices in `filter()` was deprecated in dplyr 1.1.0.
## i Please use one dimensional logical vectors instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

```

For more complex data structures and intertwined variables, exploratory analyses can become a big part of the analysis. Some techniques help detect underlying connections between variables that help to make informed decisions for follow up analyses. The interpretation of these techniques can be difficult. Among these are, for example:

- 1: Principal Component Analysis (PCA): PCA reduces the dimensionality of data by transforming it into a set of uncorrelated principal components that capture the most variance. This helps to simplify complex datasets and to identify key structures.
- 2: K-means clustering: It groups data points into a number of clusters based on their similarity, where each point belongs to the cluster with the nearest mean. This helps uncover natural groupings or segments in unlabeled data.
- 3: Exploratory Factor Analysis (EFA): EFA identifies underlying latent factors that explain the correlations among observed variables. This helps reveal the hidden structure or dimensions in the data.
- 4: Latent Class Analysis (LCA): LCA identifies unobserved subgroups in a population based on patterns in observed variables. It is useful for discovering hidden distinct groups in the data and

classifying individuals into these distinct groups.

For our purposes, we do not need to learn about PCA, k-means clustering, EFA, and LCA at this point; it might come up at a later point again.

```
#1: PCA
data$age.scaled <- scale(data$age)
data$faminc.scaled <- scale(data$family_income)
data$sleep.scaled <- scale(data$lfstl_sle)

out1_pc <- prcomp(data[c("age.scaled", "faminc.scaled", "sleep.scaled")])
data$out1_pc <- out1_pc$x[,1]

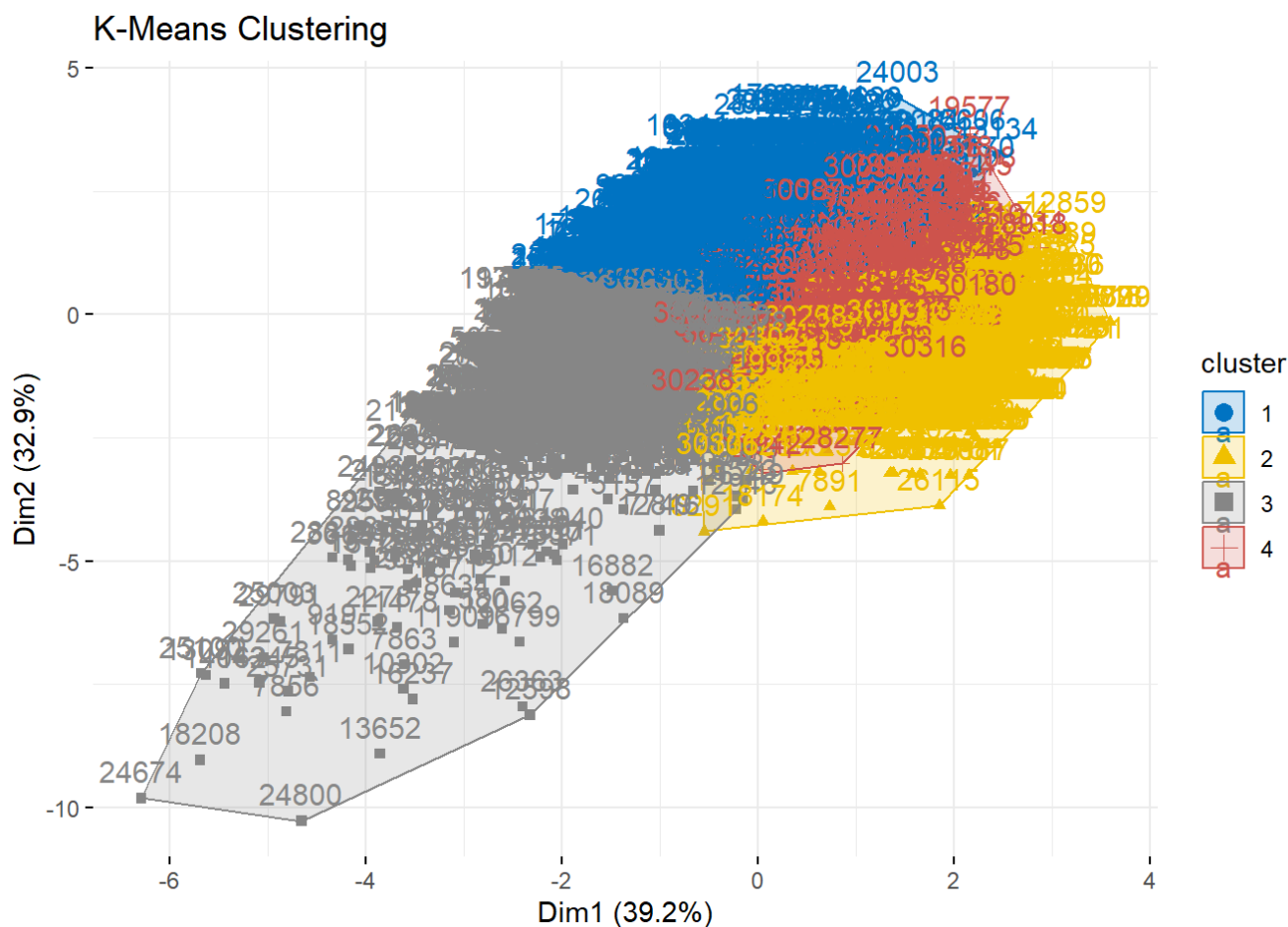
#2: k-means clustering
library(factoextra)
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WBa
```

```
#fviz_nbclust(data[c("age.scaled", "faminc.scaled", "sleep.scaled")],
#             kmeans, method = "wss") + labs(title = "Optimal k")

set.seed(123)
kmeans_clustering <- kmeans(data[c("age.scaled", "faminc.scaled", "sleep.scaled")],
                           centers = 4, nstart = 25)

fviz_cluster(kmeans_clustering,
             data = data[c("age.scaled", "faminc.scaled", "sleep.scaled")],
             ellipse.type = "convex",
             palette = "jco",
             ggtheme = theme_minimal(),
             main = "K-Means Clustering")
```



```
data$clusters <- cbind(data, cluster = kmeans_clustering$cluster)
```

```
#3: EFA
```

```
library(psych)
```

```
## Warning: package 'psych' was built under R version 4.2.3
```

```
##
```

```
## Attaching package: 'psych'
```

```
## The following object is masked from 'package:Hmisc':
```

```
##
```

```
## describe
```

```
## The following objects are masked from 'package:ggplot2':
```

```
##
```

```
## %+%, alpha
```

```
efa <- fa(r = data[c("age.scaled", "faminc.scaled", "sleep.scaled")],
         nfactors = 2, rotate = "varimax")
print(efa)
```

```
## Factor Analysis using method = minres
## Call: fa(r = data[c("age.scaled", "faminc.scaled", "sleep.scaled")],
##       nfactors = 2, rotate = "varimax")
## Standardized loadings (pattern matrix) based upon correlation matrix
##           MR1   MR2   h2   u2 com
## age.scaled    0.38  0.29 0.228 0.77 1.9
## faminc.scaled -0.36 -0.01 0.133 0.87 1.0
## sleep.scaled  0.03  0.28 0.078 0.92 1.0
##
##
##           MR1   MR2
## SS loadings    0.28 0.16
## Proportion Var    0.09 0.05
## Cumulative Var    0.09 0.15
## Proportion Explained 0.64 0.36
## Cumulative Proportion 0.64 1.00
##
## Mean item complexity = 1.3
## Test of the hypothesis that 2 factors are sufficient.
##
## df null model = 3 with the objective function = 0.03 with Chi Square = 869.59
## df of the model are -2 and the objective function was 0
##
## The root mean square of the residuals (RMSR) is 0
## The df corrected root mean square of the residuals is NA
##
## The harmonic n.obs is 30328 with the empirical chi square 0 with prob < NA
## The total n.obs was 30328 with Likelihood Chi Square = 0 with prob < NA
##
## Tucker Lewis Index of factoring reliability = 1.003
## Fit based upon off diagonal values = 1
## Measures of factor score adequacy
##
##           MR1   MR2
## Correlation of (regression) scores with factors    0.49 0.38
## Multiple R square of scores with factors          0.24 0.15
## Minimum correlation of possible factor scores      -0.51 -0.71
```

```
# 4: LCA
library(poLCA)
```

```
## Warning: package 'poLCA' was built under R version 4.2.3
```

```
## Loading required package: scatterplot3d
```

```
## Warning: package 'scatterplot3d' was built under R version 4.2.3
```

```
## Loading required package: MASS
```

```
##
## Attaching package: 'MASS'
```

```
## The following object is masked from 'package:dplyr':  
##  
##      select
```

```
set.seed(123)  
  
data_lca <- cbind(lfstl_vap,lfstl_smo,lfstl_alc) ~ 1  
lca <- polCA(data_lca, data, nclass = 3, nrep = 5)
```

```

## Model 1: llik = -41044.8 ... best llik = -41044.8
## Model 2: llik = -41044.8 ... best llik = -41044.8
## Model 3: llik = -41044.8 ... best llik = -41044.8
## Model 4: llik = -41044.8 ... best llik = -41044.8
## Model 5: llik = -41044.8 ... best llik = -41044.8
## Conditional item response (column) probabilities,
## by outcome variable, for each class (row)
##
## $lfstl_vap
##           Pr(1)  Pr(2)
## class 1:  0.4550 0.5450
## class 2:  0.1314 0.8686
## class 3:  0.0062 0.9938
##
## $lfstl_smo
##           Pr(1)  Pr(2)
## class 1:  0.8237 0.1763
## class 2:  0.5023 0.4977
## class 3:  0.0600 0.9400
##
## $lfstl_alc
##           Pr(1)  Pr(2)
## class 1:  0.9866 0.0134
## class 2:  0.9819 0.0181
## class 3:  0.7733 0.2267
##
## Estimated class population shares
##  0.2082 0.3611 0.4307
##
## Predicted class memberships (by modal posterior prob.)
##  0.102 0.3084 0.5896
##
## =====
## Fit for 3 latent classes:
## =====
## number of observations: 30328
## number of estimated parameters: 11
## residual degrees of freedom: -4
## maximum log-likelihood: -41044.8
##
## AIC(3): 82111.61
## BIC(3): 82203.12
## G^2(3): 6.16429e-08 (Likelihood ratio/deviance statistic)
## X^2(3): 5.625252e-08 (Chi-square goodness of fit)
##
## ALERT: negative degrees of freedom; respecify model
##

```

Exploratory data analysis is important to help make us better decision for follow-up inferential analyses, and should not be skipped. Data used in social science research is generally messy and has a lot of noise (outliers, strange distributions, missings), and we should take the exploratory part serious. For many analyses, this is, however, only the first part.